NORTHWESTERN UNIVERSITY

Multi-Agent Powerplant Monitoring System: Real-Time Anomaly Detection, Root Cause Analysis, and Human-in-the-Loop Review

A PROJECT REPORT

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

MASTER OF SCIENCE

Computer Engineering

By

Haoji Bian

EVANSTON, ILLINOIS

February 2026

Abstract

This report presents the design, implementation, and evaluation of a multi-agent system for powerplant asset monitoring. The system consists of four agents: a Monitor (Agent A) that detects anomalies in real-time telemetry using threshold-based methods, a Diagnosis agent (Agent B) that performs root cause analysis via LangChain ReAct with rule retrieval, a Ticket agent (Agent C) that creates review requests, and a Review agent (Agent D) that provides a human-in-the-loop interface with RAG-assisted chat. All agents communicate over MQTT. A physics-based simulator with fault injection (bearing wear, clogging, valve stuck, sensor drift, noise burst) enables reproducible evaluation. On 390 scenario runs, the system achieves 44.9% alert detection rate, 81.1% diagnosis accuracy, 83.3% fault scenario detection rate, and 11.9% healthy false positive rate. Bearing wear and valve stuck scenarios achieve 100% diagnosis accuracy; clogging and sensor override show lower accuracy. The report concludes with a discussion of limitations and future work.

Contents

Abstract **2**

List of Figures **8**

List of Tables **9**

Acknowledgments **10**

1 Introduction **11**

1.1 Motivation and Problem Statement 11

1.2 Objectives 11

1.3 Contributions 12

1.4 Report Organization 12

2 Background and Related Work **13**

2.1 Industrial IoT and Asset Monitoring 13

2.2 Anomaly Detection 14

2.3 Root Cause Analysis 14

2.4 Multi-Agent Systems 15

2.5 Retrieval-Augmented Generation 16

2.6 Vision Language Models 17

3 System Design **18**

3.1 Overview 18

3.2 Architectural Background 19

3.2.1 Design Rationale 19

3.3 Communication Links 20

3.4 LLM, Agent, and ReAct 22

3.5 RAG Design 23

3.6 Simulator 23

3.7 Agent A (Monitor) 24

3.8 Agent B (Diagnosis) 25

3.9 Agent C (Ticket) 25

3.10 Agent D (Review) 25

3.10.1 Salesforce OAuth and Case Architecture 26

3.11 Message Bus and Data Schemas 27

3.12 Database Design 27

3.13 Frontend Design 29

4 Implementation **30**

4.1 Technology Stack 30

4.1.1 Backend Stack 31

4.1.2 Optional Components 31

4.1.3 Frontend Stack 32

4.1.4 Project Structure and Configuration 32

4.2 Simulator Implementation 32

4.2.1 Physical Models 33

4.2.2 Model Execution Logic 36

4.2.3 Fault Injector 37

4.2.4 Scenario Executor 39

4.3 Agent A Implementation . . . 39
4.3.1 TelemetryBuffer . . . 39
4.3.2 ThresholdDetector . . . 41
4.3.3 Startup Configuration . . . 43
4.4 Agent B Implementation . . . 43
4.4.1 ReAct Agent Creation . . . 43
4.4.2 Tools . . . 44
4.4.3 Answer Parsing and Output Schema . . . 44
4.4.4 Diagnosis Flow . . . 45
4.5 Agent C Implementation . . . 46
4.6 Agent D Implementation . . . 46
4.6.1 RAG Indexing . . . 46
4.6.2 RAG and Chat Tools . . . 47
4.6.3 Approve/Reject Flow and Chat Streaming . . . 47
4.7 Database and Vector Store . . . 49
4.7.1 SQLite Schema Overview . . . 49
4.7.2 Embedding Pipeline . . . 49
4.8 Deployment . . . 51
4.8.1 Backend Deployment (EC2) . . . 51
4.8.2 Database Deployment (Backend) . . . 52
4.8.3 Frontend Deployment (S3, CloudFront, DNS) . . . 52

5 Evaluation **54**
5.1 Evaluation Setup . . . 54
5.1.1 Scenarios . . . 54
5.1.2 Metrics . . . 55
5.2 Alert Detection Results . . . 55
5.3 Diagnosis Accuracy . . . 56

5.4 Scenario Matrix 57
5.5 Token Usage and Latency 59
5.6 Discussion 60
5.7 Improvement Opportunities 60

6 Conclusion **61**
6.1 Summary 61
6.2 Future Work 62

A Database Schema **66**

B Scenario JSON Example **75**

C Rule Document Example **77**

D Project Links **79**

List of Figures

3.1 System architecture. Data flows left to right via MQTT pub/sub. SQLite stores telemetry, alerts, diagnosis, and feedback. 18

3.2 Complete system architecture: frontend (React), backend (Simulator + Agents A–D), message buses (MQTT, optional Kafka), database (SQLite), RAG, and optional Salesforce (Case creation on approve). HTTP for API/frontend; MQTT for streaming events; Kafka for Agent B diagnosis queue. 21

3.3 ReAct loop: Thought → Action → Observation. The LLM reasons, calls a tool, observes the result, and iterates. 22

3.4 RAG flow: indexing (left) and retrieval (right). Documents are embedded and stored; queries retrieve similar vectors for context. 23

3.5 Agent A (Monitor) architecture: MQTT subscription → TelemetryBuffer → ThresholdDetector → MQTT publish. 24

3.6 Agent B (Diagnosis) architecture: MQTT → Kafka (optional) → ReAct agent with tools → MQTT publish. 25

3.7 Agent D (Review) architecture: React frontend → FastAPI; API uses SQLite, RAG, ReAct chat (SSE), review queue, and optional Salesforce Case creation on approve. 26

3.8 Salesforce OAuth and Case flow: (1) Agent D obtains access token via password or client credentials flow; (2) creates Case via REST POST on approve; (3) fetches Case picklists via Describe API for form prefill. 27

5.1 Evaluation summary: scenario runs, detection rate, diagnosis accuracy, healthy false positive rate, fault scenario detection rate. 55
5.2 Alert detection rate by signal (%). 56
5.3 Diagnosis accuracy by root cause (%). 57
5.4 Confusion matrix: expected (rows) vs. predicted (columns). 58
5.5 Scenario matrix: detection rate vs. diagnosis accuracy per fault scenario. 58
5.6 Steps and tokens: correct vs. incorrect diagnoses. 59
5.7 Average token usage by scenario (k tokens). 59

List of Tables

3.1 Design rationale for key architectural choices. 20
3.2 Communication protocols and their roles. 21
3.3 Agent B (Diagnosis) tools. 23
3.4 Agent D (Review) RAG and VLM tools. 24
3.5 Fault types and physical effects. 24
3.6 Message payload schemas (key fields). 28
3.7 MQTT topic convention. 28
3.8 Database tables. 29

4.1 Technology stack by category. 30
4.2 Slope thresholds for trend-based alerts. 42
4.3 Agent B (Diagnosis) tools. 44
4.4 Agent D (Review) tools. 48
4.5 SQLite tables and main columns (full schema in Appendix A). 50
4.6 Backend components (EC2). 51
4.7 Deployment summary: backend vs. frontend. 53

5.1 Alert detection by signal. 56
5.2 Diagnosis accuracy by root cause. 57

Acknowledgments

I would like to thank my advisor, Professor Qi Zhu, for his guidance and support throughout this project. I am grateful to Simon Zhan, my PhD mentor, for his valuable advice and feedback. I also thank David Zaretsky, Director of the MS Program in Computer Engineering, for his support of the graduate program.

Chapter 1

Introduction

1.1 Motivation and Problem Statement

Industrial Internet of Things (IoT) asset monitoring requires real-time processing of sensor telemetry, timely fault detection, and interpretable root cause analysis (RCA) to support maintenance decisions. Traditional approaches often rely on black-box machine learning models that lack transparency, or on manual ticket creation that delays response. Furthermore, safety-critical applications demand human-in-the-loop review before automated actions are executed.

This report addresses the following research question: Can a multi-agent system combining rule-based anomaly detection, large language model (LLM)-based diagnosis with ReAct (Reasoning and Acting) reasoning, and retrieval-augmented generation (RAG) achieve reliable fault diagnosis with interpretable evidence?

1.2 Objectives

The primary objectives of this project are:

1. Design and implement a four-agent pipeline: Monitor $\rightarrow$ Diagnosis $\rightarrow$ Ticket $\rightarrow$ Review, with each agent fulfilling a distinct role.

2. Integrate threshold-based anomaly detection, ReAct-based diagnosis with tool use, and RAG for context retrieval in the review interface.

3. Evaluate detection rate, diagnosis accuracy, and healthy false positive rate on fault-injected scenarios using a reproducible evaluation framework.

1.3 Contributions

The main contributions of this report are:

- An end-to-end multi-agent architecture for powerplant monitoring, with agents communicating via MQTT (Message Queuing Telemetry Transport).
- A rule-plus-LLM hybrid diagnosis approach with evidence traceability through ReAct reasoning and tool calls.
- A reproducible evaluation framework with fault injection and scenario-driven testing.

1.4 Report Organization

Chapter 2 reviews related work in industrial IoT, anomaly detection, root cause analysis, multi-agent systems, and RAG. Chapter 3 presents the system design, including architectural background, LLM/ReAct/RAG design, and component specifications. Chapter 4 describes the implementation. Chapter 5 presents the evaluation results. Chapter 6 concludes and outlines future work.

Chapter 2

Background and Related Work

2.1 Industrial IoT and Asset Monitoring

Industrial IoT enables real-time sensor streams from physical assets such as pumps, motors, and bearings [1, 2, 3]. Lee et al. [1] describe a cyber-physical systems architecture for Industry 4.0, where manufacturing systems integrate sensing, computation, and actuation for predictive maintenance. Wang et al. [2] survey deep-learning-enabled predictive maintenance in industrial IoT, highlighting methods for anomaly detection, remaining useful life estimation, and fault prognosis. Zhu et al. [3] provide a comprehensive survey of intelligent predictive maintenance for industrial equipment, covering data-driven and model-based approaches.

SCADA (Supervisory Control and Data Acquisition) systems and predictive maintenance frameworks aim to detect anomalies before failures occur, reducing unplanned downtime and maintenance costs. Common fault types in rotating machinery include bearing wear, pipe clogging, valve malfunction, and sensor drift. These faults manifest as distinct patterns in telemetry signals—e.g., bearing wear increases vibration and temperature; clogging reduces flow and increases pressure differential—which motivates the design of multi-signal detection and diagnosis pipelines.

2.2 Anomaly Detection

Anomaly detection identifies patterns that do not conform to expected behavior [4, 5]. Chandola et al. [4] provide a broad survey of anomaly detection techniques, categorizing them into statistical, machine learning, and hybrid approaches. Ahmed et al. [5] focus on network anomaly detection but discuss principles applicable to general time-series data.

Statistical methods such as Z-score and threshold-based detection with sliding windows are widely used for real-time telemetry. These methods are interpretable, require minimal training data, and can be tuned for specific signals. Threshold-based detection checks whether a signal exceeds a configured limit; duration checks help avoid false positives from transient spikes. Slope-based detection identifies trends (e.g., rising temperature over time) that may indicate gradual degradation.

Machine learning approaches, including deep learning, offer flexibility but often sacrifice interpretability. Deep models can learn complex patterns from raw or aggregated data but may require large labeled datasets and lack transparency for safety-critical applications. Hybrid approaches that combine rule-based thresholds with learned models are increasingly explored for industrial IoT monitoring.

In industrial settings, baseline or normal-profile modeling establishes expected ranges for each signal under healthy conditions [3]. A sliding-window buffer maintains recent samples for computing statistics (mean, variance, slope) over short intervals. Duration checks require that a threshold violation persist for a minimum time before triggering an alert, reducing false positives from transient noise. Wang et al. [2] note that combining multiple detection criteria—e.g., threshold, slope, and duration—improves robustness for real-time telemetry streams.

2.3 Root Cause Analysis

Root cause analysis (RCA) links observed symptoms to underlying faults [6, 7]. Isermann [6] provides a foundational treatment of fault-diagnosis systems, from fault detection to fault tolerance.

Yin et al. [7] compare data-driven fault diagnosis methods on the Tennessee Eastman benchmark, demonstrating trade-offs between model-based and data-driven approaches.

Rule-based systems provide interpretability: domain experts encode symptom-to-fault mappings in rules or decision trees, enabling traceable diagnoses. Data-driven methods, such as support vector machines or neural networks, can capture complex patterns but may be less interpretable. Zhang et al. [8] apply fault tree analysis for power electronic systems; Chen and Patton [9] discuss robust model-based fault diagnosis for dynamic systems.

Recent work on LLM-based reasoning with tool use [10, 11] enables multi-step inference with evidence traceability. Yao et al. [10] introduce ReAct: interleaving reasoning (Thought) and acting (tool calls) in language models. Schick et al. [11] show that language models can learn to use tools for external knowledge retrieval. This paradigm is well-suited for diagnosis: the LLM can query rules, telemetry, and past alerts, then produce structured output with root cause and evidence.

Beyond ReAct, chain-of-thought prompting and tree-based search extend LLM reasoning. Yao et al. [12] propose Tree of Thoughts, which explores multiple reasoning paths and backtracks when a path fails, enabling deliberate problem solving. Such methods complement tool use by improving the quality of multi-step inference when the solution space is large or ambiguous.

2.4 Multi-Agent Systems

Multi-agent systems coordinate autonomous entities through message passing and pub/sub architectures [13, 14]. Wooldridge [13] defines agents as autonomous entities that perceive their environment and act to achieve goals. Russell and Norvig [14] discuss agent architectures, including reactive agents and agents with internal state.

In industrial IoT, a multi-agent design allows separation of concerns: one agent monitors telemetry, another diagnoses faults, another creates tickets, and another supports human review. This modularity improves scalability and fault isolation—a failure in one agent does not necessarily halt the entire pipeline.

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe protocol suitable for industrial IoT [15, 16]. OASIS [15] defines the MQTT v3.1.1 standard. Recent work such as MQTT-I [16] analyzes MQTT deployments and protocol-level issues, including end-to-end data integrity when brokers may not be fully trusted. Topics such as `telemetry/{asset_id}` and `alerts/{asset_id}` enable loose coupling between agents.

2.5 Retrieval-Augmented Generation

RAG enriches language model context with retrieved documents [17, 18]. Lewis et al. [17] introduce retrieval-augmented generation for knowledge-intensive NLP tasks, combining a dense retriever with a sequence-to-sequence model. Borgeaud et al. [18] demonstrate that retrieving from large external corpora improves language model performance on factual and reasoning tasks.

Vector search and semantic retrieval support domain knowledge integration for diagnosis and review. Embedding models (e.g., sentence-transformers) map text to dense vectors; similarity search retrieves relevant past diagnoses, rules, or alerts. This enables the review agent to answer questions such as "What similar past diagnoses exist?" or "Which rules apply to this symptom?" without relying solely on the LLM's parametric knowledge.

RAG is particularly valuable when domain knowledge is dynamic (e.g., new rules or feedback) or when the model's training cutoff predates recent events. For powerplant monitoring, indexing alerts, diagnoses, rules, feedback, and chat messages into a vector store allows the agent to ground its responses in context-specific evidence.

Practical RAG implementations rely on embedding models and vector indexes. Reimers and Gurevych [19] introduce Sentence-BERT, which produces semantically meaningful sentence embeddings via siamese BERT networks; the sentence-transformers library provides pre-trained models such as all-MiniLM-L6-v2 (384 dimensions) for efficient retrieval. Vector databases store embeddings and support similarity search; sqlite-vec [20] extends SQLite with vector similarity search, enabling lightweight, file-based RAG without a separate vector store service.

2.6 Vision Language Models

Vision language models (VLMs) extend LLMs to process both images and text, enabling multimodal reasoning over visual inputs [21, 22, 23]. Li et al. [21] present BLIP for unified vision-language understanding and generation; Liu et al. [22] introduce visual instruction tuning for aligning vision encoders with language models. Zhang et al. [23] survey recent advances in multimodal large language models. In industrial monitoring, VLMs can analyze visualization images (e.g., 3D pump renderings or dashboard screenshots) and produce textual descriptions or anomaly reports, complementing sensor-based diagnosis with visual evidence.

Chapter 3

System Design

3.1 Overview

The system comprises five main components: a Simulator, Agent A (Monitor), Agent B (Diagnosis), Agent C (Ticket), and Agent D (Review). Data flows from telemetry to alerts, diagnosis, tickets, and human feedback. Figure 3.1 shows the high-level architecture. The Simulator generates telemetry and publishes to MQTT; Agent A detects anomalies and publishes alerts; Agent B diagnoses root causes via ReAct and publishes diagnosis reports; Agent C creates review requests; Agent D provides the human-in-the-loop interface with a React frontend, RAG-assisted chat, and approve/reject workflow.

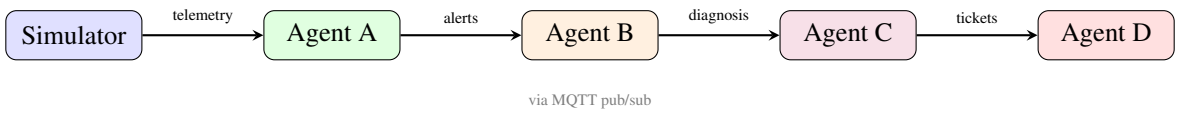

Figure 3.1: System architecture. Data flows left to right via MQTT pub/sub. SQLite stores telemetry, alerts, diagnosis, and feedback.

3.2 Architectural Background

The multi-agent design provides separation of concerns, scalability, and fault isolation [13]. Each agent is stateless and processes events independently; no shared mutable state is required beyond the message bus and database. Pub/sub via MQTT [15, 16] enables loose coupling and asynchronous event flow. Unlike request-response architectures, agents do not block waiting for replies; they publish events and consume from topics asynchronously. This design allows horizontal scaling (e.g., multiple Agent A instances for different assets) and fault isolation—a failure in one agent does not necessarily halt the pipeline.

3.2.1 Design Rationale

Table 3.1 summarizes why each architectural choice was made. **Multi-agent:** Splitting Monitor, Diagnosis, Ticket, and Review into separate agents allows independent development, deployment, and scaling; a failure in diagnosis does not block alert detection. **MQTT:** Chosen over HTTP for streaming telemetry because it is lightweight, supports pub/sub natively, and is widely used in industrial IoT; telemetry arrives at high frequency and MQTT avoids per-message HTTP overhead. **HTTP:** Used for frontend–backend communication and REST APIs because it is the standard for web applications; the frontend needs request-response semantics for user actions. **Kafka (optional):** Agent B's ReAct diagnosis can take several seconds; without a queue, the MQTT callback would block. Kafka decouples alert reception from LLM inference. **ReAct:** Pure prompt-based diagnosis lacks evidence traceability; ReAct's interleaving of thought and tool calls lets the LLM query rules and telemetry, producing structured output with citations [10]. **RAG:** Domain knowledge (rules, past diagnoses) changes over time; RAG grounds the agent in retrieved context rather than relying solely on parametric knowledge [17]. **SQLite:** Chosen for simplicity—no separate database server, file-based, sufficient for single-node deployment. **React:** Component-based SPA with good support for SSE (chat streaming) and REST APIs.

Figure 3.2 shows the complete system architecture including frontend, backend services, mes-

Table 3.1: Design rationale for key architectural choices.

Choice	Alternative	Rationale
Multi-agent	Monolith	Separation of concerns, fault isolation, horizontal scaling
MQTT	HTTP polling	Lightweight pub/sub, IoT standard, low latency for streaming
HTTP	WebSocket only	Standard for REST API, frontend request-response
Kafka (opt.)	Sync MQTT	Decouples alert handler from LLM latency
ReAct	Pure prompt	Multi-step reasoning, tool use, evidence traceability
RAG	Parametric only	Dynamic domain knowledge, grounding in context
SQLite	PostgreSQL	Simplicity, no separate server, file-based
React	Server-rendered	SPA, SSE for streaming, component reuse
Salesforce (opt.)	Other CRM	Integrates with enterprise ticketing; Case creation on approve

sage buses, database, RAG, and external access. The browser accesses the system via DNS (e.g., app.powerplantagent.com) or EC2 public IP. The React frontend (port 3000 in development) proxies API requests to Agent D (8005) and Simulator (8001). Backend services expose HTTP APIs for health checks and control; Agent B optionally uses Kafka for diagnosis request queuing. All agents read/write SQLite; Agent D uses the RAG vector store (sqlite-vec) for semantic search. Optional external integrations: Agent D can create a Salesforce Case when a reviewer approves a diagnosis; Agent C may create tickets in GitHub or other systems when configured.

3.3 Communication Links

The system uses HTTP, MQTT, Kafka (optional), and outbound HTTP for external integrations. Table 3.2 summarizes their roles.

HTTP: Each backend service exposes an HTTP API (FastAPI). The frontend proxies `/api` to Agent D (8005) and `/simulator` to Simulator (8001). External access uses DNS or EC2 public

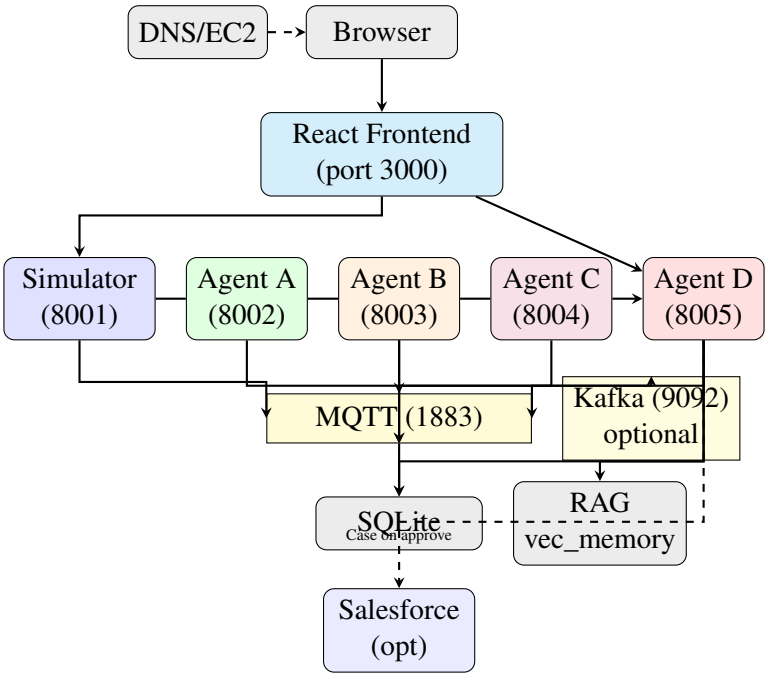

Figure 3.2: Complete system architecture: frontend (React), backend (Simulator + Agents A–D), message buses (MQTT, optional Kafka), database (SQLite), RAG, and optional Salesforce (Case creation on approve). HTTP for API/frontend; MQTT for streaming events; Kafka for Agent B diagnosis queue.

Table 3.2: Communication protocols and their roles.

Protocol	**Port**	**Description**
HTTP	8001, 8002–8005	REST API; health checks; frontend proxy; Simulator control
MQTT	1883	Pub/sub: telemetry, alerts, diagnosis, tickets, feedback
Kafka	9092 (optional)	Agent B diagnosis queue; decouples MQTT from LLM
HTTP (outbound)	443	Agent D: Salesforce REST API (Case creation on approve, optional)

IP; ports 8001 and 8005 are typically exposed for web and API access.

MQTT: All agents communicate via MQTT pub/sub. The Simulator publishes telemetry to `telemetry/{asset_id}`; Agent A subscribes, detects anomalies, and publishes to `alerts/{asset_id}`; Agent B subscribes to alerts and publishes diagnosis to `diagnosis/{asset_id}`; Agent C subscribes to diagnosis and publishes tickets to `tickets/{asset_id}`; Agent D publishes feedback to `feedback/{asset_id}`.

Kafka: When configured (`KAFKA_BOOTSTRAP_SERVERS`), Agent B uses Kafka to queue diagnosis requests. The MQTT callback enqueues alerts to the `diagnosis-queue` topic; a consumer runs the ReAct agent asynchronously. This decouples MQTT from LLM latency and avoids blocking the alert handler.

External HTTP: Agent D optionally calls the Salesforce REST API when a reviewer approves a diagnosis and checks "Create Salesforce Case." The Case is created with Subject (root cause), Description, and picklist fields; a ticket row is stored locally. Configuration uses `SALESFORCE_DOMAIN` and access token or OAuth credentials.

3.4 LLM, Agent, and ReAct

The diagnosis agent uses the ReAct paradigm [10]: interleaving Thought, Action, and Observation. The LLM reasons over tool outputs and produces structured JSON with root cause, confidence, and evidence. Figure 3.3 illustrates the ReAct loop. The agent perceives (alerts), reasons (Thought), acts (tool calls), observes (tool results), and repeats until it produces a final answer. Table 3.3 lists the tools available to Agent B.

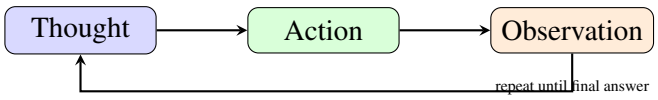

Figure 3.3: ReAct loop: Thought → Action → Observation. The LLM reasons, calls a tool, observes the result, and iterates.

Table 3.3: Agent B (Diagnosis) tools.

Tool	Description
query_rules	Search diagnosis rules by keywords (signal names, fault types)
query_telemetry	Get recent telemetry for an asset (time range, limit)
query_alerts	Get recent alerts for an asset

3.5 RAG Design

RAG indexes alerts, diagnoses, rules, feedback, and chat messages into a vector store [17]. Agent D uses semantic search (sqlite-vec, all-MiniLM-L6-v2) for query_similar_diagnoses, query_similar_rules, and related tools [19, 20]. Figure 3.4 shows the indexing and retrieval flow. Table 3.4 lists the RAG and VLM tools available to Agent D.

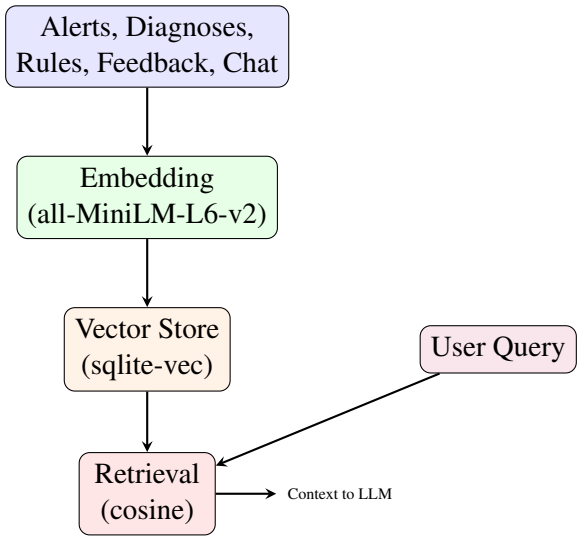

Figure 3.4: RAG flow: indexing (left) and retrieval (right). Documents are embedded and stored; queries retrieve similar vectors for context.

3.6 Simulator

The simulator models pump, piping, and bearing physics. Fault injection supports bearing_wear, clogging, valve_stuck, sensor_drift, and noise_burst. Scenarios are defined in JSON scripts. Table 3.5 summarizes the fault types and their physical effects.

Table 3.4: Agent D (Review) RAG and VLM tools.

Tool	Description
query_similar_diagnoses	Semantic search for similar past diagnoses
query_similar_rules	Semantic search for relevant rules
query_similar_alerts	Semantic search for similar alerts
query_similar_feedback	Semantic search for past feedback
query_similar_chat	Semantic search for past chat
analyze_image_with_vlm	Analyze image (e.g., 3D pump render) via VLM

Table 3.5: Fault types and physical effects.

Fault Type	Physical Effect	Signals Affected
bearing_wear	Increased friction, vibration	vibration_rms, bearing_temp_c
clogging	Flow restriction, pressure rise	flow_m3h, pressure_bar
valve_stuck	Valve position mismatch	valve_open_pct, flow_m3h
sensor_drift	Bias in sensor reading	temp_c, pressure_bar
noise_burst	Transient spike	any signal

3.7 Agent A (Monitor)

Agent A subscribes to telemetry, applies threshold and slope-based detection with duration checks, and publishes alerts to MQTT. A sliding-window buffer maintains recent samples for computing statistics. Threshold checks require a signal to exceed a configured limit; duration checks require the violation to persist for a minimum time before triggering an alert, reducing false positives from transient noise. Slope-based detection identifies trends (e.g., rising temperature) that may indicate gradual degradation. Figure 3.5 shows the internal structure.

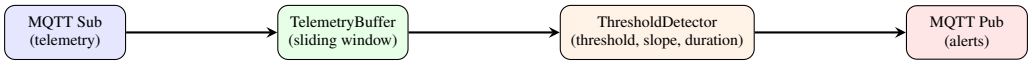

Figure 3.5: Agent A (Monitor) architecture: MQTT subscription → TelemetryBuffer → ThresholdDetector → MQTT publish.

3.8 Agent B (Diagnosis)

Agent B subscribes to alerts, runs a ReAct agent with tools (Table 3.3), retrieves rules from Markdown documents, and publishes diagnosis reports. The diagnosis report schema includes root_cause, confidence, impact, recommended_actions, and evidence. The agent parses the LLM's final JSON output and publishes to the diagnosis topic. When Kafka is enabled, alerts are queued and a consumer runs diagnosis asynchronously. Figure 3.6 shows the architecture.

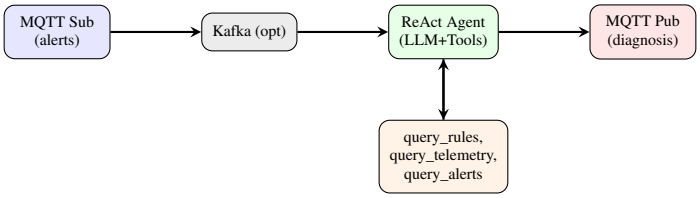

Figure 3.6: Agent B (Diagnosis) architecture: MQTT → Kafka (optional) → ReAct agent with tools → MQTT publish.

3.9 Agent C (Ticket)

Agent C creates review requests from diagnoses. No LLM is used; it is a rule-based relay. For each diagnosis, it creates a review_request with status="pending" and queues it for Agent D approval.

3.10 Agent D (Review)

Agent D provides a human-in-the-loop interface with RAG tools (Table 3.4), approve/reject workflow, and optional external integrations. When a reviewer approves a diagnosis, they may optionally create a Salesforce Case via REST API; the Case form is pre-filled with Subject (root cause), Description (diagnosis summary), Priority, Status, and other picklist fields fetched from the Salesforce org. Configuration uses `SALESFORCE_DOMAIN` and either `SALESFORCE_ACCESS_TOKEN` or OAuth (client_id, client_secret, username, password). If not configured, approval is recorded locally only. The frontend is a React web app with pages for Review Queue, Alerts, Sensors, Chat,

and Scenarios. The Chat page streams ReAct steps via Server-Sent Events (SSE). The agent can call analyze_image_with_vlm to analyze 3D pump renderings or dashboard screenshots [21, 22]. Figure 3.7 shows the architecture.

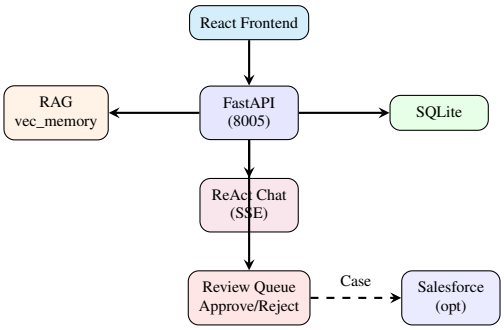

Figure 3.7: Agent D (Review) architecture: React frontend → FastAPI; API uses SQLite, RAG, ReAct chat (SSE), review queue, and optional Salesforce Case creation on approve.

3.10.1 Salesforce OAuth and Case Architecture

When Salesforce integration is enabled, Agent D authenticates via OAuth 2.0 [24] and creates Cases through the REST API [25]. This subsection describes the OAuth connection flow and the Case object architecture.

OAuth Connection. The system supports two configuration methods. *Method A (Access Token):* Provide `SALESFORCE_DOMAIN` and a pre-obtained `SALESFORCE_ACCESS_TOKEN`; the token is obtained manually (e.g., via curl) using the password grant. *Method B (Password Flow):* Provide `SALESFORCE_DOMAIN`, `SALESFORCE_CLIENT_ID`, `SALESFORCE_CLIENT_SECRET`, `SALESFORCE_USERNAME`, and `SALESFORCE_PASSWORD` (login password concatenated with the user's Security Token). The connector requests an access token from `https://{domain}/services/oauth2/token` with `grant_type=password`. A Connected App must be created in Salesforce Setup (App Manager) with OAuth enabled; required scopes include `api` and `refresh_token`. Optionally, client credentials flow is supported if the Connected App is configured for it. Figure 3.8 illustrates the OAuth and Case creation flow.

Case Object Architecture. Case is a standard Salesforce object for customer support [25].

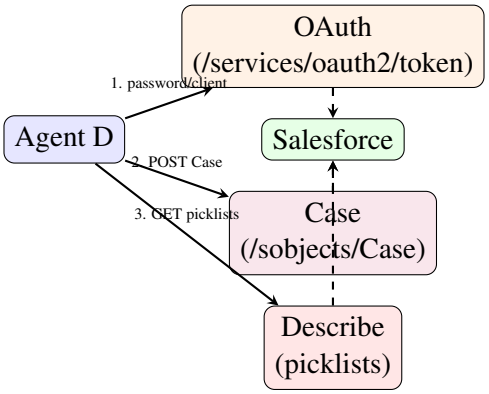

Figure 3.8: Salesforce OAuth and Case flow: (1) Agent D obtains access token via password or client credentials flow; (2) creates Case via REST POST on approve; (3) fetches Case picklists via Describe API for form prefill.

The connector sends a POST to `/services/data/v58.0/sobjects/Case` with JSON payload: `Subject` (root cause, prefixed with `[asset_id]`), `Description` (diagnosis summary), `Status`, `Priority`, `Origin`, `Type`, and `Reason`. Status, Priority, Origin, Type, and Reason are picklist fields; valid values are fetched from `/sobjects/Case/describe` so the frontend can render dropdowns. On success, the API returns the Case Id; Agent D stores a ticket row locally with `ticket_id` and `url` (Lightning record URL). The `TicketConnector` interface allows swapping Salesforce for other CRM systems (e.g., ServiceNow, Zendesk) with minimal changes.

3.11 Message Bus and Data Schemas

MQTT topics follow the convention in Table 3.7. Telemetry, AlertEvent, DiagnosisReport, Ticket, and Feedback schemas define the message payloads. Table 3.6 summarizes the key fields for each payload type. Each agent publishes to downstream topics and subscribes to upstream topics.

3.12 Database Design

SQLite stores telemetry, alerts, diagnosis, vision images, tickets, review requests, chat sessions, chat messages, chat steps, and feedback. Table 3.8 summarizes the main tables. The vec_memory

Table 3.6: Message payload schemas (key fields).

Schema	**Source**	**Key Fields**
Telemetry	Simulator	ts, plant_id, asset_id, signals (pressure_bar, flow_m3h, temp_c, bearing_temp_c, vibration_rms, rpm, motor_current_a, valve_open_pct), truth (fault, severity)
AlertEvent	Agent A	ts, plant_id, asset_id, severity (warning/critical), alerts (signal, score, method, evidence)
DiagnosisReport	Agent B	ts, plant_id, asset_id, root_cause, confidence, impact, recommended_actions, evidence
Ticket	Agent C	ts, plant_id, asset_id, ticket_system, ticket_id, url
Feedback	Agent D	ts, plant_id, asset_id, ticket_id, review_decision (approved/rejected/edited), final_root_cause, notes

Table 3.7: MQTT topic convention.

Topic	**Description**
telemetry/{asset_id}	Raw telemetry from Simulator
alerts/{asset_id}	Alerts from Agent A
diagnosis/{asset_id}	Diagnosis reports from Agent B
tickets/{asset_id}	Tickets from Agent C
feedback/{asset_id}	Human feedback from Agent D
vision/{asset_id}	Vision image paths (optional)

virtual table (sqlite-vec) stores 384-dimensional embeddings for RAG; it is indexed by cosine similarity.

Table 3.8: Database tables.

Table	**Description**
telemetry	Sensor readings from Simulator
alerts	Alerts from Agent A
diagnosis	Diagnosis reports from Agent B
vision_images	Image paths from Simulator
vision_analysis	VLM analysis results
tickets	Tickets from Agent C
review_requests	Pending review queue
chat_sessions, chat_messages, chat_steps	Chat persistence
feedback	Human approval/rejection
vec_memory	Vector embeddings (RAG)

3.13 Frontend Design

The Agent D frontend is a React single-page application with the following pages: Review Queue (pending diagnoses, approve/reject), Alerts (alert list with filtering), Sensors (real-time telemetry display), Chat (ReAct agent with SSE streaming), and Scenarios (simulator control). The frontend communicates with the Agent D backend via REST API and SSE for chat streaming. The layout uses a sidebar for navigation and a main content area that switches based on route.

Chapter 4

Implementation

This chapter describes the implementation of the multi-agent powerplant monitoring system. Each component is built with Python 3.11 and integrates via MQTT and REST APIs. The implementation follows the architecture described in Chapter 3.

4.1 Technology Stack

The system uses a cohesive technology stack chosen for rapid development, interoperability, and reproducibility. The backend is implemented in Python; the Agent D frontend is a React single-page application. Table 4.1 summarizes the main components by category.

Table 4.1: Technology stack by category.

Category	Technologies
Runtime	Python 3.11
Web framework	FastAPI, pydantic, pydantic-settings, uvicorn
Message bus	MQTT (paho-mqtt), Mosquitto broker; Kafka (kafka-python, optional)
Agent framework	LangChain, LangGraph, langchain-openai
RAG / vector search	sqlite-vec, sentence-transformers
Streaming	sse-starlette (Server-Sent Events)
HTTP client	httpx
Frontend	React 18, Vite, React Router, Axios, Recharts, react-markdown
Evaluation	matplotlib
Optional / external	Salesforce (OAuth, Case API), GitHub API, pyvista (3D), anthropic (Claude Vision)

4.1.1 Backend Stack

Python 3.11 provides modern type hints and performance improvements. FastAPI [26] offers async support, automatic OpenAPI documentation, and Pydantic validation for request/response schemas. Configuration is managed via `pydantic-settings` with environment variables and an optional `.env` file. MQTT via paho-mqtt [15] enables lightweight pub/sub for telemetry and alerts; Kafka (kafka-python) is optionally used as a diagnosis queue for Agent B when high throughput is needed. LangChain [27] and LangGraph [28] implement the ReAct agent loop [10] with tool calling; `langchain-openai` provides the LLM interface (compatible with DeepSeek and OpenAI). The RAG pipeline uses sqlite-vec [20] for in-process vector storage and sentence-transformers [19, 29] (all-MiniLM-L6-v2) for embeddings, avoiding external API calls. Agent D streams chat responses via sse-starlette; httpx is used for outbound HTTP requests. Matplotlib generates evaluation charts for detection rates and diagnosis accuracy.

4.1.2 Optional Components

Several optional components extend the system when configured. **Salesforce:** Agent D uses the Salesforce REST API [25] with OAuth 2.0 [24] (password flow or access token) to create Cases when a reviewer approves a diagnosis; the `SalesforceConnector` in `shared_lib/integrations/salesforce.py` implements the `TicketConnector` interface. **GitHub:** Agent C can create GitHub Issues as tickets when `GITHUB_TOKEN` and `GITHUB_REPO` are set. **pyvista:** The simulator optionally renders 3D pump visualizations for the vision pipeline. **anthropic:** Claude Vision (Claude 3) is used for `analyze_image_with_vlm` when multimodal analysis is enabled [21, 22]; the VLM analyzes 3D renderings or dashboard screenshots. All optional components are configured via environment variables and degrade gracefully when not present.

4.1.3 Frontend Stack

The Agent D review dashboard is a React 18 single-page application built with Vite. React Router DOM handles client-side routing (Review Queue, Alerts, Sensors, Chat, Scenarios). Axios performs REST API calls to the Agent D backend. Recharts renders real-time telemetry charts on the Sensors page. React Markdown with remark-gfm displays chat responses and ReAct reasoning steps with proper markdown formatting (tables, code blocks). The frontend is served as static assets by the Agent D FastAPI server after `npmrunbuild`; in development, Vite provides hot module replacement. The build output targets modern browsers (browserslist: `>0.2%,notdead`).

4.1.4 Project Structure and Configuration

The project is organized as a monorepo: `simulator-service/` (simulator), `agent-monitor/`, `agent-diagnosis/`, `agent-ticket/`, `agent-review/` (backend and frontend), `shared_lib/` (models, config, db, embeddings, vector_db, vector_indexing), `scripts/` (init_db, start_backend_ec2), and `evaluation/`. Each service runs as a separate process; they share `shared_lib` and communicate via MQTT and REST. Optional integrations (GitHub for Agent C tickets, Salesforce for Case creation on approval) are configured via environment variables when needed.

4.2 Simulator Implementation

The simulator produces realistic pump telemetry with configurable faults. It consists of four physical models (pump, pipe, bearing, motor), a fault injector, and a scenario executor that drives the simulation loop from JSON definitions. The models are based on first-principles physics [30]; faults modify model parameters or outputs to produce detectable anomalies.

4.2.1 Physical Models

Each model implements equations from fluid mechanics, electrical engineering, or mechanical vibration. The models are coupled: the pump and pipe determine the operating point (Q,H); the pump power drives the motor current and bearing load; the bearing yields vibration and temperature.

Pump Model

The `PumpModel` uses a quadratic head-flow curve and affinity laws [30]. At rated conditions (n_N, Q_N, H_N), the pump head is

$$H_{\text{pump}}(Q,n) = H_{\text{shutoff}}(n) - a \cdot Q^2, \quad H_{\text{shutoff}}(n) = H_{\text{shutoff},N} \cdot (n/n_N)^2, \tag{4.1}$$

where n is rpm, Q is flow (m^3/h), and $a = (H_{\text{shutoff},N} - H_N)/Q_N^2$. Affinity laws scale flow and head with speed: $Q \propto n$, $H \propto n^2$. Valve opening limits maximum flow: $Q_{\max} = Q_N \cdot (n/n_N) \cdot (\text{valve}/100)$. Efficiency is parabolic around the best-efficiency point:

$$\eta(Q) = \eta_N \cdot \left(1 - \left(\frac{Q - Q_{\text{opt}}}{Q_{\text{opt}}}\right)^2\right), \quad Q_{\text{opt}} = Q_N \cdot (n/n_N). \tag{4.2}$$

Shaft power follows $P_{\text{shaft}} = \rho g Q H/\eta$ (with $\rho = 1000\,\text{kg/m}^3$, $g = 9.81\,\text{m/s}^2$). The operating point is found by solving $H_{\text{pump}}(Q) = H_{\text{system}}(Q)$ via binary search. Listing 4.1 shows `compute_head` and `solve_operating_point`.

Listing 4.1: PumpModel.compute_head and solve_operating_point (simulator-service/models/pump_model.py)

```
def compute_head(self, Q: float, rpm: float) -> float:
    n_ratio = rpm / self.n_N
    H_shutoff_scaled = self.H_shutoff * (n_ratio ** 2)
    H = H_shutoff_scaled - self.a_coeff * (Q ** 2)
```

```
    return max(0.0, H)


def solve_operating_point(self, rpm, valve_open_pct, system_head_func
   ):
    _, Q_max = self.compute_head_flow_curve(rpm, valve_open_pct)
    Q_low, Q_high = 0.0, Q_max
    for _ in range(50):
        Q_mid = (Q_low + Q_high) / 2.0
        H_pump = self.compute_head(Q_mid, rpm)
        H_system = system_head_func(Q_mid)
        if abs(H_pump - H_system) < 0.01:
            return Q_mid, H_pump
        if H_pump > H_system: Q_low = Q_mid
        else: Q_high = Q_mid
    Q_final = (Q_low + Q_high) / 2.0
    return Q_final, self.compute_head(Q_final, rpm)
```

Pipe System Model

The `PipeSystemModel` uses the Darcy-Weisbach equation [31] for friction loss. System head is

$$H_{\text{system}}(Q) = R \cdot Q^2_{\text{m}^3/\text{s}} + H_{\text{static}}, \tag{4.3}$$

where $R = f(L/D)/(2gA^2) + K/(2gA^2)$ combines pipe friction ($f \approx 0.02$) and fittings loss (K). Flow is converted from m^3/h to m^3/s for the quadratic term. Clogging increases resistance: $R_{\text{current}} = R_{\text{base}} \cdot \text{clogging_factor}$. Listing 4.2 shows the implementation.

Listing 4.2: PipeSystemModel.compute_system_head and set_clogging_resistance

```
def compute_system_head(self, Q: float) -> float:
    Q_m3s = Q / 3600.0
```

```
    H_dynamic = self.R_current * (Q_m3s ** 2)
    return H_dynamic + self.H_static

def set_clogging_resistance(self, clogging_factor: float):
    self.clogging_factor = max(1.0, clogging_factor)
    self.R_current = self.R_base * self.clogging_factor
```

Bearing Model

The `BearingModel` tracks wear level $d \in [0,1]$ (0 = healthy, 1 = failure), following vibration-based condition monitoring principles [32]. Wear accumulates linearly: $d \leftarrow d + \dot{d} \cdot \Delta t$. Vibration RMS (mm/s) increases non-linearly with wear:

$$v = v_{\text{base}} \cdot (1 + 2d + 5d^3) \cdot (1 + 0.2(\lambda - 1)), \tag{4.4}$$

where λ is load factor ($P_{\text{shaft}}/10$ kW). Temperature rises with heat generation (increased friction) and degraded thermal dissipation:

$$\Delta T = \text{heat_gen} \cdot \text{thermal_resistance} \cdot 20, \quad \text{heat_gen} = \lambda(1+3d), \quad \text{thermal_resistance} = \frac{0.5}{1+0.5d}. \tag{4.5}$$

Listing 4.3 shows `compute_vibration` and `compute_bearing_temperature`.

Listing 4.3: BearingModel.compute_vibration and compute_bearing_temperature

```
def compute_vibration(self, load_factor: float = 1.0) -> float:
    wear_effect = 1.0 + self.wear_level * 2.0 + (self.wear_level **
        3) * 5.0
    load_effect = 1.0 + 0.2 * (load_factor - 1.0)
    return max(0.1, self.base_vibration_rms * wear_effect *
        load_effect)
```

```
def compute_bearing_temperature(self, load_factor: float = 1.0) ->
   float:
    heat_generation_factor = load_factor * (1.0 + self.wear_level *
       3.0)
    thermal_resistance = 0.5 / (1.0 + self.wear_level * 0.5)
    temp_rise = heat_generation_factor * thermal_resistance * 20.0
    base_temp_rise = self.base_bearing_temp - self.ambient_temp
    return max(self.ambient_temp, self.ambient_temp + base_temp_rise
       + temp_rise)
```

Motor Model

The `MotorModel` converts shaft power to motor current. Electrical power is $P_{\text{elec}} = P_{\text{shaft}}/\eta_{\text{motor}}$. For three-phase AC:

$$I = \frac{P_{\text{elec}}}{\sqrt{3} \cdot U \cdot \cos\phi} + I_{\text{no-load}}. \tag{4.6}$$

Default parameters: $U = 400\,\text{V}$, $\eta_{\text{motor}} = 0.92$, $\cos\phi = 0.85$, $I_{\text{no-load}} = 5\,\text{A}$.

4.2.2 Model Execution Logic

The `ScenarioExecutor.step(dt)` runs each time step in order: (1) update setpoints (rpm, valve from scenario); (2) `fault_injector.update_faults` (e.g., bearing wear, clogging); (3) apply valve faults (valve_stuck); (4) solve $H_{\text{pump}}(Q) = H_{\text{system}}(Q)$ for (Q,H); (5) compute η, P_{shaft}, I_{motor}; (6) compute bearing vibration and temperature; (7) map to sensor signals (pressure $\approx 0.1 \cdot H$ bar, flow, temp, etc.); (8) apply sensor faults (drift, override, noise_burst); (9) add Gaussian sensor noise; (10) return `Telemetry`. Listing 4.4 shows the core loop.

Listing 4.4: ScenarioExecutor.step core logic (simulator-service/scenarios/scenario_executor.py)

```
def step(self, dt: float) -> Optional[Telemetry]:
```

```
    self._update_setpoints()
    self.fault_injector.update_faults(self.current_time, dt)
    valve_actual = self.fault_injector.apply_valve_faults(self.
       valve_open_pct)
    Q, H = self.pump.solve_operating_point(
        self.rpm, valve_actual, self.pipe.compute_system_head
    )
    eta = self.pump.compute_efficiency(Q, self.rpm)
    P_shaft = self.pump.compute_power(Q, H, eta)
    I_motor = self.motor.compute_current(P_shaft)
    load_factor = P_shaft / 10.0
    vibration_rms = self.bearing.compute_vibration(load_factor)
    bearing_temp = self.bearing.compute_bearing_temperature(
       load_factor)
    true_signals = {"pressure_bar": H*0.1, "flow_m3h": Q, ...}
    sensor_signals = self.fault_injector.apply_sensor_faults(
       true_signals)
    self._add_sensor_noise(sensor_signals)
    self.current_time += dt
    return Telemetry(signals=..., truth=...)
```

4.2.3 Fault Injector

The `FaultInjector` maintains a mapping from fault type names to fault classes and applies fault-specific updates each time step. Listing 4.5 shows the `inject_fault` method.

Listing 4.5: FaultInjector.inject_fault (simulator-service/faults/fault_injector.py)

```
def inject_fault(self, fault_type: str, start_time: float, params:
   Dict[str, Any]):
```

```
    fault_map = {
        "bearing_wear": BearingWearFault,
        "clogging": CloggingFault,
        "valve_stuck": ValveStuckFault,
        "sensor_drift": SensorDriftFault,
        "sensor_stuck": SensorStuckFault,
        "sensor_override": SensorOverrideFault,
        "noise_burst": NoiseBurstFault,
    }
    fault_class = fault_map.get(fault_type)
    if fault_class:
        fault = fault_class(start_time, params)
        self.active_faults.append(fault)
    else:
        raise ValueError(f"Unknown fault type: {fault_type}")
```

Each fault type implements `update(current_time,dt,models)` to modify physical models. For example, `BearingWearFault` increases bearing wear over time, which raises vibration and temperature (Listing 4.6).

Listing 4.6: BearingWearFault.update (simulator-service/faults/fault_types.py)

```
def update(self, current_time: float, dt: float, models: Dict[str,
   Any]):
    if current_time >= self.start_time:
        self.active = True
        self.elapsed_time += dt
        bearing_model = models.get("bearing")
        if bearing_model:
            bearing_model.update_wear(self.rate_per_sec, dt)
```

4.2.4 Scenario Executor

The `ScenarioExecutor` is initialized with a scenario dictionary (from JSON) and pump config. It creates `PumpModel`, `PipeSystemModel`, `BearingModel`, and `MotorModel` from config; registers them with the fault injector; and injects faults from `scenario["faults"]`. A scenario JSON includes `name`, `duration_sec`, `initial_conditions` (rpm, valve_open_pct), `faults` (type, start_time_sec, params), `setpoints` (time-ordered rpm/valve changes), and optional `seed` for reproducible sensor noise. The main simulation loop (in `simulator-service/main.py`) runs `step(dt)` at `simulator_frequency_hz` (default 1 Hz), publishes telemetry to MQTT, and optionally writes to the database.

4.3 Agent A Implementation

Agent A subscribes to MQTT telemetry, detects anomalies via thresholds and slopes [4, 3], and publishes alerts. The data flow is: MQTT callback `on_telemetry` receives telemetry → `buffer.push(telemetry)` → `detector.detect(telemetry,buffer)` → if alert and not in cooldown: insert to DB, `index_alert` for RAG, publish to MQTT. Two core components are `TelemetryBuffer` and `ThresholdDetector`.

4.3.1 TelemetryBuffer

`TelemetryBuffer` maintains a per-asset sliding window of recent telemetry (default `window_sec`=120 s, `max_points_per_asset`=200). On each `push`, it appends a `BufferedPoint` and trims points older than `window_sec`. `compute_stats(asset_id,signal,window_sec)` returns mean μ, std σ, and slope:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} v_i, \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(v_i-\mu)^2}, \quad \text{slope} = \frac{v_n - v_1}{t_n - t_1} \quad \text{(units/sec)}. \tag{4.7}$$

`duration_above_threshold(asset_id,signal,threshold,side,window_sec)` returns the seconds the signal has been above (side="high") or below (side="low") the threshold within the window; a single matching point is counted as 0.5 s to allow fast triggers. Listing 4.7 shows `compute_stats` and `duration_above_threshold`.

Listing 4.7: TelemetryBuffer.compute_stats and duration_above_threshold

```
def compute_stats(self, asset_id, signal, window_sec=None) -> Dict[
    str, Any]:
    points = self.get_window(asset_id, signal, window_sec)
    if len(points) < 2:
        return {"mean": None, "std": None, "slope": None}
    values = [v for _, v in points]
    mean = sum(values) / len(values)
    std = (sum((x - mean)**2 for x in values) / len(values)) ** 0.5
    dt = points[-1][0].timestamp() - points[0][0].timestamp()
    slope = (values[-1] - values[0]) / dt if dt > 0 else 0.0
    return {"mean": mean, "std": std, "slope": slope}

def duration_above_threshold(self, asset_id, signal, threshold, side=
    "high", window_sec=None) -> float:
    points = self.get_window(asset_id, signal, window_sec)
    total_sec, n_matching = 0.0, 0
    for i in range(len(points)):
        t, v = points[i]
        above = (side == "high" and v >= threshold) or (side == "low"
            and v <= threshold)
        if above:
            n_matching += 1
            if i + 1 < len(points):
```

```
                total_sec += (points[i+1][0] - t).total_seconds()
    if n_matching >= 1 and total_sec < 0.5:
        total_sec = 0.5
    return total_sec
```

4.3.2 ThresholdDetector

`ThresholdDetector` checks signals against `DEFAULT_THRESHOLDS` (Listing 4.8), implementing statistical threshold-based detection [4]. Threshold types: **low-side** (e.g., `flow_m3h`: warning_low, critical_low); **high-side** (e.g., `vibration_rms`, `bearing_temp_c`: warning, critical); **range** (e.g., `rpm`: min, max). Critical severity takes precedence over warning. Each `AlertDetail` includes `signal`, `score`, `method` (threshold/slope/combination), and `evidence` (value, threshold, duration, mean, std, slope). Signals in `FAST_DURATION_SIGNALS` (`temp_c`, `rpm`, `valve_flow_mismatch`) use `min_dur`=0.5 s; others use `monitor_min_duration_sec` (default 5 s) to reduce false positives.

Listing 4.8: DEFAULT_THRESHOLDS (agent-monitor/detection/threshold_detector.py)

```
DEFAULT_THRESHOLDS = {
    "vibration_rms": {"warning": 2.8, "critical": 18.0},
    "bearing_temp_c": {"warning": 55.0, "critical": 85.0},
    "pressure_bar": {"warning_high": 7.5, "critical_high": 25.0},
    "motor_current_a": {"warning_high": 34.0, "critical_high": 45.0},
    "flow_m3h": {"warning_low": 48.0, "critical_low": 45.0},
    "rpm": {"min": 2650.0, "max": 3150.0},
}
```

Table 4.2 lists `DEFAULT_SLOPE_THRESHOLDS` for trend-based alerts. Slope is in units per second; `side`="low" means alert when slope falls below the threshold (e.g., flow dropping).

When a buffer is provided, the detector only emits a threshold alert if the signal has sustained

Table 4.2: Slope thresholds for trend-based alerts.

Signal	Warning	Critical	Side
vibration_rms	0.008	0.08	high
bearing_temp_c	0.38	0.6	high
flow_m3h	−0.85	−5.0	low
pressure_bar	0.25	1.0	high
motor_current_a	0.95	1.5	high

the breach for at least `min_dur` seconds (Listing 4.9).

Listing 4.9: Duration check in ThresholdDetector.detect

```
min_dur = 0.5 if signal_name in FAST_DURATION_SIGNALS else self.
   min_duration_sec
if buffer and min_dur > 0:
    thr_val = rule.get("critical") or rule.get("critical_high") or
       rule.get("critical_low")
    if thr_val is not None:
        dur = buffer.duration_above_threshold(
            telemetry.asset_id, signal_name, thr_val, side, self.
               window_sec
        )
        if dur < min_dur:
            continue  # Not sustained long enough
```

Valve-flow mismatch is a composite rule: valve $\geq 65\%$ and flow $\leq 68\,\mathrm{m}^3/\mathrm{h}$ (healthy valve 70% yields flow ≈ 70). It uses `duration_valve_flow_mismatch` and triggers after 0.5 s sustained. **Cooldown:** per (asset_id, signal) a 60 s cooldown prevents re-alerting the same breach. On alert: `shared_db.insert_alert`, `index_alert` for RAG, `AlertPublisher.publish` to MQTT topic `alerts/{asset_id}`.

4.3.3 Startup Configuration

On startup, Agent A creates `TelemetryBuffer(window_sec=120,max_points_per_asset=200)`, `ThresholdDetector(min_duration_sec=settings.monitor_min_duration_sec,window_sec=60)`, subscribes to `telemetry/#`, and initializes `AlertPublisher` with `settings.mqtt_topic_alerts`. The `monitor_min_duration_sec` is configurable (e.g., 2 s for evaluation to trigger alerts faster).

4.4 Agent B Implementation

Agent B runs a ReAct [10] diagnosis agent that consumes alerts (from Kafka or MQTT), invokes the LLM with tools, parses the final answer, and publishes a `DiagnosisReport` to MQTT. When `KAFKA_BOOTSTRAP_SERVERS` is set, alerts are enqueued to the `diagnosis-queue` topic and a worker runs diagnosis asynchronously; otherwise the MQTT callback runs `run_diagnosis` synchronously. A per-(asset_id, signals) cooldown (`diagnosis_cooldown_sec`, default 60 s) prevents re-diagnosing the same alert pattern.

4.4.1 ReAct Agent Creation

The agent is built with LangGraph's [28] `create_react_agent` (Listing 4.10). The LLM prefers DeepSeek (OpenAI-compatible) with `temperature=0` for deterministic reasoning; OpenAI GPT-4o-mini is a fallback. The system prompt instructs the agent to: (1) read the alert summary; (2) use `query_rules` with signal names or symptoms; (3) optionally use `query_telemetry` or `compute_slope` to confirm trends; (4) output a JSON diagnosis.

Listing 4.10: create_diagnosis_agent (agent-diagnosis/agent/agent.py)

```
def create_diagnosis_agent():
    from langgraph.prebuilt import create_react_agent
    llm = _make_llm()  # DeepSeek or OpenAI
    tools = get_diagnosis_tools()
```

```
    return create_react_agent(llm, tools)
```

4.4.2 Tools

Table 4.3 lists the tools available to the diagnosis agent. Tool-augmented LLMs [11] enable the agent to query external knowledge (rules, telemetry, alerts) before producing a diagnosis. Rules are stored as Markdown files in `diagnosis_rules_path` (default `agent-diagnosis/rules`); `query_rules` scans for keyword matches and returns up to five documents. `compute_slope` computes trend (slope per second) from the telemetry window for a given signal.

Table 4.3: Agent B (Diagnosis) tools.

Tool	**Parameters**	**Description**
query_rules	keywords	Search diagnosis rules by signal names, fault types, or symptoms. Returns matching Markdown content (max 5).
query_telemetry	asset_id, since_ts, until_ts, limit	Query recent telemetry from DB. Optional time range. Returns formatted rows (P, F, T, BT, Vib, RPM, I, Valve).
query_alerts	asset_id, limit, since_ts, until_ts	Query recent alerts for an asset. Returns id, ts, severity, signal, score, method, evidence.
get_telemetry_window	asset_id, window_sec	Fetch telemetry over last N seconds. For trend analysis.
compute_slope	asset_id, signal, window_sec	Compute slope (change per second), mean, std for a signal. Detects gradual increase (bearing wear) or sudden drop (clogging).

4.4.3 Answer Parsing and Output Schema

The agent outputs a JSON object with `root_cause` (bearing_wear, clogging, valve_stuck, sensor_drift, unknown), `confidence` (0–1), `impact` (low, medium, high), `recommended_actions`

(list), and `evidence` (list of rule objects), aligning with structured fault-diagnosis output conventions [6]. `_parse_final_answer` extracts this from the last message, handling markdown code blocks and brace-balanced JSON (Listing 4.11).

Listing 4.11: _parse_final_answer (agent-diagnosis/agent/agent.py)

```
def _parse_final_answer(text: str) -> Optional[Dict[str, Any]]:
    for block in re.finditer(r'```(?:json)?\s*([\s\S]*?)```', text):
        try:
            return json.loads(block.group(1).strip())
        except json.JSONDecodeError:
            continue
    start = text.find('{')
    while start >= 0:
        # Find brace-balanced JSON containing "root_cause"
        ...
        if "root_cause" in obj:
            return obj
    return None
```

4.4.4 Diagnosis Flow

`run_diagnosis(alert_payload)` builds the user prompt from `_build_alert_summary` (asset_id, ts, severity, signal list with scores and evidence), invokes the agent with `recursion_limit` (default 40), and converts the parsed JSON into a `DiagnosisReport`. On success: `shared_db.insert_diagnosis` (including alert_id, actual_steps, token counts), `index_diagnosis` for RAG, `DiagnosisPublisher.publish` to MQTT topic `diagnosis/{asset_id}`.

4.5 Agent C Implementation

Agent C subscribes to MQTT topic `diagnosis/#`. On each diagnosis payload, it calls `shared_db.insert_review_request(diagnosis_id,plant_id,asset_id,ts,status=‘‘pending’’)`. A per-asset cooldown (`ticket_cooldown_sec`, default 30 s) prevents duplicate review requests. No LLM is used; it is a rule-based relay. Agent D polls `review_requests` from the database for the review queue.

4.6 Agent D Implementation

Agent D provides the review queue, approve/reject workflow, and a chat interface with RAG-backed tools [17]. It is built with LangGraph's `create_react_agent` [28] and `get_review_tools()`; the system prompt instructs the agent to help operators review pending diagnoses, inspect alerts and telemetry, and recommend approve/reject decisions. RAG indexing runs automatically when alerts, diagnoses, feedback, tickets, or chat messages are created.

4.6.1 RAG Indexing

`shared_lib.vector_indexing` provides `index_diagnosis`, `index_alert`, `index_feedback`, `index_ticket`, `index_chat_message`, `index_vision_analysis`, and `index_rules`. Each is wrapped with `_safe_index` so indexing failures do not break the main flow. Embeddings use sentence-transformers [19, 29] and the vector store is sqlite-vec [20]. Listing 4.12 shows `index_diagnosis`.

Listing 4.12: index_diagnosis (shared_lib/vector_indexing.py)

```
@_safe_index
def index_diagnosis(diagnosis_id: int, diagnosis_data: Dict[str, Any
    ]) -> Optional[int]:
    root_cause = diagnosis_data.get("root_cause", "")
    confidence = diagnosis_data.get("confidence", 0.0)
```

```
    recommended_actions = diagnosis_data.get("recommended_actions",
        [])
    evidence = diagnosis_data.get("evidence", [])
    actions_str = ", ".join(recommended_actions) if isinstance(
        recommended_actions, list) else str(recommended_actions)
    evidence_str = json.dumps(evidence) if isinstance(evidence, (list
        , dict)) else str(evidence)
    text = f"Root cause: {root_cause}\nConfidence: {confidence:.2f}\n
        ..."
    return add_text_to_vector_db(
        text=text.strip(),
        doc_type="diagnosis",
        doc_id=diagnosis_id,
        extra_metadata={"asset_id": diagnosis_data.get("asset_id", ""
            ), "root_cause": root_cause},
    )
```

4.6.2 RAG and Chat Tools

Table 4.4 lists the tools available to the review agent. DB tools query relational data; RAG tools perform semantic search over indexed diagnoses, alerts, rules, feedback, and chat history [17]. `query_similar_diagnoses` calls `search_text_in_vector_db` with `filter_type="diagnosis"` and returns similarity scores ($1 - \text{L2 distance}$). `analyze_image_with_vlm` invokes a vision-language model on 3D pump renderings or dashboard screenshots [21, 22].

4.6.3 Approve/Reject Flow and Chat Streaming

The API exposes `POST/api/review/{review_id}/approve` and `POST/api/review/{review_id}/reject`. On approve: `shared_db.update_review_request_status(review_id`

Table 4.4: Agent D (Review) tools.

Tool	Parameters	Description
query_review_requests	status, asset_id, limit	List pending review requests. Filter by status (e.g. pending) and asset_id.
query_diagnosis	diagnosis_id	Get full diagnosis details (root_cause, confidence, evidence, recommended_actions).
query_alerts	asset_id, limit, since_ts, until_ts	Query recent alerts for an asset. Optional time range.
query_telemetry	asset_id, since_ts, until_ts, limit	Query recent telemetry for an asset.
query_vision_images	asset_id, limit	List vision image records (ts, asset_id, image_path) for VLM input.
query_rules	keywords	Search diagnosis rules by keyword (e.g. vibration, bearing, clogging).
query_salesforce_cases	asset_id, created_since, severity, keywords, limit	Query Salesforce Cases (when configured). SOQL search on Subject, Description, Priority.
query_similar_diagnoses	query, limit	RAG: semantic search over past diagnoses. Returns diagnosis_id, similarity, root_cause.
query_similar_alerts	query, limit	RAG: semantic search over past alerts. Returns alert_id, similarity, signal, severity.
query_similar_rules	query, limit	RAG: semantic search over diagnosis rules. Returns rule_name, similarity, content preview.
query_similar_feedback	query, limit	RAG: semantic search over past feedback. Returns feedback_id, similarity, review_decision.
query_similar_chat	query, limit	RAG: semantic search over chat history. Returns message_id, similarity, answer preview.
analyze_image_with_vlm	image_path, question	Call VLM to describe an image or answer a question (e.g. “Any anomalies?”).

,‘‘approved’’), insert_feedback, index_feedback. When “Create Salesforce Case” is checked, approve-with-case calls run_approve_assistant (LLM analyzes diagnosis, fetches similar Cases, suggests form fields) then SalesforceConnector.create_case. Chat responses are streamed via SSE (sse-starlette); the frontend receives incremental tokens and ReAct steps (thought, tool_call, tool_result) for transparency. The React frontend includes pages for Review Queue, Alerts, Sensors, Chat, and Scenarios.

4.7 Database and Vector Store

The system uses a single SQLite database for relational data and sqlite-vec [20] for vector embeddings. The schema is created by scripts/init_db.py. Full CREATETABLE definitions and indexes are given in Appendix A.

4.7.1 SQLite Schema Overview

Table 4.5 summarizes the relational tables and their roles. The Simulator writes telemetry; Agent A writes alerts; Agent B writes diagnosis; Agent C writes review_requests; Agent D reads review_requests, writes feedback, tickets (on approve with Case), chat_sessions, chat_messages, chat_steps, and optionally vision_analysis. Indexes on (asset_id,ts) and (ts,asset_id) support time-range queries. The vec_memory virtual table (sqlite-vec) stores 384-dimensional embeddings for RAG; it is created by init_db.py when the extension is available.

4.7.2 Embedding Pipeline

shared_lib.embeddings provides EmbeddingModel (sentence-transformers [19, 29], default all-MiniLM-L6-v2, 384 dimensions). add_text_to_vector_db encodes text, prepares metadata (type, id, text preview), and inserts into vec_memory. search_text_in_vector_db encodes the query and runs similarity search with optional filter_type. Distance is L2; similarity is computed as $1 - \text{distance}$ for display.

Table 4.5: SQLite tables and main columns (full schema in Appendix A).

Table	**Source**	**Main columns**
telemetry	Simulator	id, ts, plant_id, asset_id, pressure_bar, flow_m3h, temp_c, bearing_temp_c, vibration_rms, rpm, motor_current_a, valve_open_pct, fault
alerts	Agent A	id, ts, plant_id, asset_id, severity, signal, score, method, evidence
diagnosis	Agent B	id, ts, plant_id, asset_id, root_cause, confidence, impact, recommended_actions, evidence, alert_id
vision_images	Simulator	id, ts, asset_id, image_path
vision_analysis	Agent D (VLM)	id, ts, asset_id, image_path, description, anomalies_detected
tickets	Agent D	id, ts, asset_id, ticket_id, title, body, status, diagnosis_id, url
review_requests	Agent C	id, diagnosis_id, plant_id, asset_id, ts, status, resolved_at
chat_sessions	Agent D	id, created_at, updated_at, preview
chat_messages	Agent D	id, session_id, role, content, tool_calls
chat_steps	Agent D	id, message_id, step_type, step_order, tool_name, tool_args, content
feedback	Agent D	id, ts, asset_id, ticket_id, review_decision, final_root_cause, notes

4.8 Deployment

Deployment is split into backend (AWS EC2) and frontend (S3, CloudFront, DNS). The backend runs all services on one EC2 instance; the database (SQLite) and vector store (sqlite-vec) are co-located on the same instance. The frontend is built as static assets and served via S3 and CloudFront, with DNS pointing the application domain to CloudFront.

4.8.1 Backend Deployment (EC2)

Prerequisites on the EC2 instance: Python 3.11+ with a virtual environment, `pipinstall-rrequirements.txt`, and Docker (for the MQTT broker). The startup script `scripts/start_backend_ec2.sh` starts components in dependency order. Table 4.6 lists each backend component, its port, and how it is run.

Table 4.6: Backend components (EC2).

Component	Port	Process	Notes
MQTT (Mosquitto)	1883	Docker	`dockercomposeup-dmosquitto`; pub/sub for telemetry, alerts, diagnosis.
Simulator	8001	`simulator-service/main.py`	Publishes telemetry to MQTT; optional DB write.
Agent A (Monitor)	8002	`agent-monitor/main.py`	Subscribes to telemetry, publishes alerts.
Agent B (Diagnosis)	8003	`agent-diagnosis/main.py`	Subscribes to alerts; optional Kafka queue.
Agent C (Ticket)	8004	`agent-ticket/main.py`	Subscribes to diagnosis, writes `review_requests`.
Agent D (Review)	8005	`agent-review/main.py`	FastAPI: review API, chat SSE, optional static frontend.

Logs are written to `logs/` (e.g., `simulator.log`, `agent_a.log`) and process IDs to `.pids/`. The script runs a health check (curl to `/health` or `/status`) after startup. EC2 security groups must allow inbound TCP 8001 (Simulator), 8005 (Agent D API), and optionally 1883 (MQTT) for

external clients. Configuration (MQTT host, API keys, SQLITE_PATH, etc.) is read from .env or environment variables.

4.8.2 Database Deployment (Backend)

The database is deployed on the same EC2 instance as the backend. SQLite is file-based; no separate database server is required. Deployment steps: (1) Ensure SQLITE_PATH is set (e.g., data/monitoring.db) or use the default from .env; (2) run pythonscripts/init_db.py once (or pythonscripts/init_db.py<path>) to create the schema and indexes; (3) optionally run scripts/backup_db.sh for backups. The vec_memory virtual table (sqlite-vec) is created by init_db.py when the extension is available; all backend processes share the same SQLite file. For production at scale, the database could be moved to a managed service (e.g., RDS) with minimal code changes to the connection string.

4.8.3 Frontend Deployment (S3, CloudFront, DNS)

The frontend is a React single-page application (Vite build). Deployment is separate from the backend so that static assets can be served from a CDN.

1. **Build:** Run npmrunbuild in agent-review/frontend. Output goes to dist/ (or build/).

2. **S3:** Upload the contents of dist/ to an S3 bucket configured for static website hosting or for use as a CloudFront origin. Ensure the bucket policy allows CloudFront (or public) read access to the objects.

3. **CloudFront:** Create a CloudFront distribution with the S3 bucket as the origin. Configure default root object (e.g., index.html) and error responses (e.g., 404 → index.html) for client-side routing. Optionally restrict viewer protocol policy to HTTPS and attach an ACM certificate for a custom domain.

4. **DNS:** Point the application domain (e.g., `app.powerplantagent.com`) to the CloudFront distribution via a CNAME (or Route 53 alias). Users then access the frontend at `https://app.powerplantagent.com`; the frontend must be configured with the backend API base URL (e.g., `https://api.powerplantagent.com` or the EC2 endpoint) so that API requests are sent to the correct host.

Table 4.7 summarizes the deployment split. The backend API (Agent D on port 8005) can be exposed via a separate DNS name (e.g., API gateway or a second CloudFront origin with EC2 or ALB as origin) so that the browser calls `https://api....` instead of the raw EC2 IP; otherwise, CORS must allow the frontend origin.

Table 4.7: Deployment summary: backend vs. frontend.

Layer	**Platform**	**Details**
Backend (all services + DB)	EC2	One instance; `start_backend_ec2.sh`; SQLite + sqlite-vec on instance; `.env` for config.
Frontend (static)	S3 + CloudFront	Build → upload to S3 → CloudFront origin; HTTPS + custom domain via DNS.
DNS	Route 53 / CNAME	Domain points to CloudFront (frontend); optional second name for API (e.g., ALB or EC2).

Chapter 5

Evaluation

5.1 Evaluation Setup

The evaluation measures alert detection quality, diagnosis accuracy, and reasoning efficiency from recorded scenario runs. In total, 390 scenario runs are used (210 healthy, 180 fault). The pipeline consists of: (1) start all services (MQTT, Simulator, Agent A/B/C/D); (2) run `scripts/run_alert_eval.py`, which executes health and fault scenarios in sequence and records runs to `evaluation/scenario_runs.jsonl`; (3) run `evaluation/run_evaluation.py` to compute metrics from the database and scenario runs; (4) run `evaluation/build_report.py` to generate charts. Alerts and diagnoses are matched to each run by time window; detection requires at least one alert within the run window, and diagnosis accuracy is computed per diagnosis against the run's expected root cause.

5.1.1 Scenarios

Seven scenario types are used: `healthy_baseline` (no fault, 210 runs), `bearing_wear_eval`, `clogging_eval`, `valve_flow_mismatch_eval`, `sensor_drift_eval_temp_override`, `rpm_eval_override` (both map to sensor_override), and `noise_burst_eval`. Each fault scenario runs 30 times; healthy runs 210 times (6 cycles $\times$ 35). Fault scenarios inject bearing wear, pipe clogging,

valve stuck, sensor drift/override, or transient noise bursts.

5.1.2 Metrics

Detection rate: fraction of scenario runs that produced at least one alert. **Fault scenario detection rate:** same, excluding healthy runs. **Diagnosis accuracy:** fraction of diagnoses whose predicted `root_cause` matches the expected root cause for the scenario. **Healthy false positive rate:** fraction of `healthy_baseline` runs that produced any alert. **Token usage and latency:** average ReAct steps, total tokens, and alert-to-diagnosis latency per scenario. Figure 5.1 summarizes the main KPIs.

Evaluation Summary

Metric	Value
Scenario Runs	390
Alert Detection Rate	44.9%
Diagnosis Accuracy	81.1%
Healthy False Positive	11.9%
Alert Accuracy (TP+TN)/total	85.9%
Fault Scenario Detection (excl. h	83.3%

Figure 5.1: Evaluation summary: scenario runs, detection rate, diagnosis accuracy, healthy false positive rate, fault scenario detection rate.

5.2 Alert Detection Results

Overall detection rate across all runs is 44.9%; excluding healthy runs, fault scenario detection rate is 83.3%. Table 5.1 and Figure 5.2 show detection by signal. `bearing_temp_c`, `flow_m3h`, `pressure_bar`, `motor_current_a`, `valve_flow_mismatch`, `temp_c`, and `rpm` achieve 100% detection on their expected scenarios; `vibration_rms` achieves 50% (30 of 60 expected runs). `noise_burst_eval` produces no alerts (0% detection) because the transient spike does not meet the sustained duration threshold (`min_duration_sec`). Healthy false positive rate is 11.9% (25 of 210 runs); all

false positives are from `bearing_temp_c`, likely due to ambient or startup temperature variation.

Table 5.1: Alert detection by signal.

Signal	**Expected runs**	**Hits**	**Detection rate (%)**
bearing_temp_c	30	30	100
flow_m3h	30	30	100
pressure_bar	30	30	100
motor_current_a	30	30	100
valve_flow_mismatch	30	30	100
temp_c	30	30	100
rpm	30	30	100
vibration_rms	60	30	50

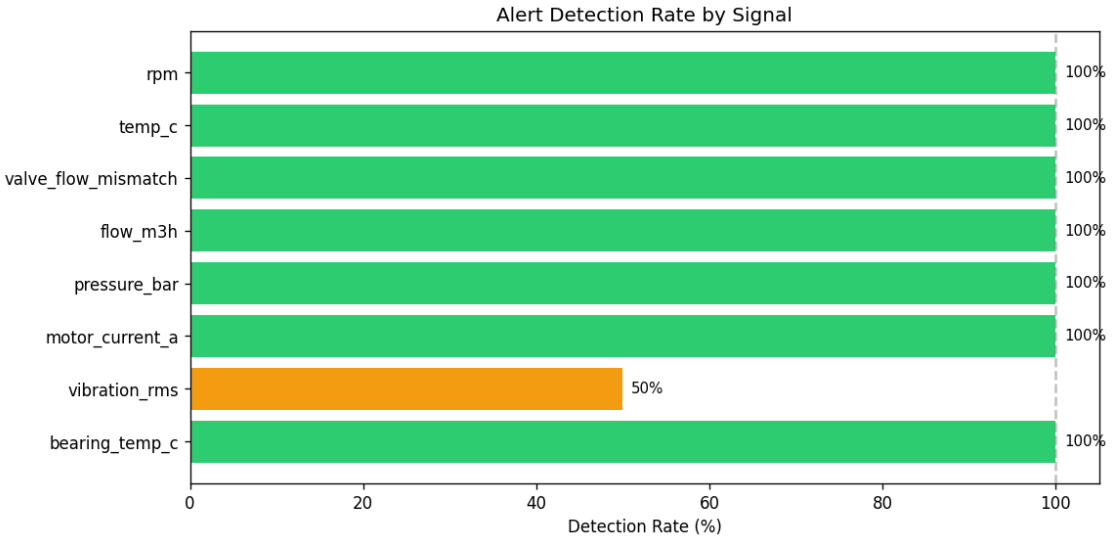

Figure 5.2: Alert detection rate by signal (%).

5.3 Diagnosis Accuracy

Across 196 diagnoses, 159 are correct (81.1% accuracy). Table 5.2 and Figure 5.3 show per-root-cause accuracy. `bearing_wear` and `valve_stuck` achieve 100%; `clogging` 73.7%; `sensor_override` 53.3%; `none` (healthy) 66.7%. Figure 5.4 shows the confusion matrix. Clogging is confused with `valve_stuck` and `bearing_wear`; valve_stuck with `clogging`; sensor_override with `sensor_drift`, `bearing_wear`, and `clogging`; healthy (`none`) is misdiagnosed as `bearing`

_wear, clogging, or sensor_drift.

Table 5.2: Diagnosis accuracy by root cause.

Root cause	Count	Correct	Accuracy (%)
bearing_wear	78	78	100.0
valve_stuck	27	27	100.0
clogging	19	14	73.7
sensor_override	60	32	53.3
none	12	8	66.7

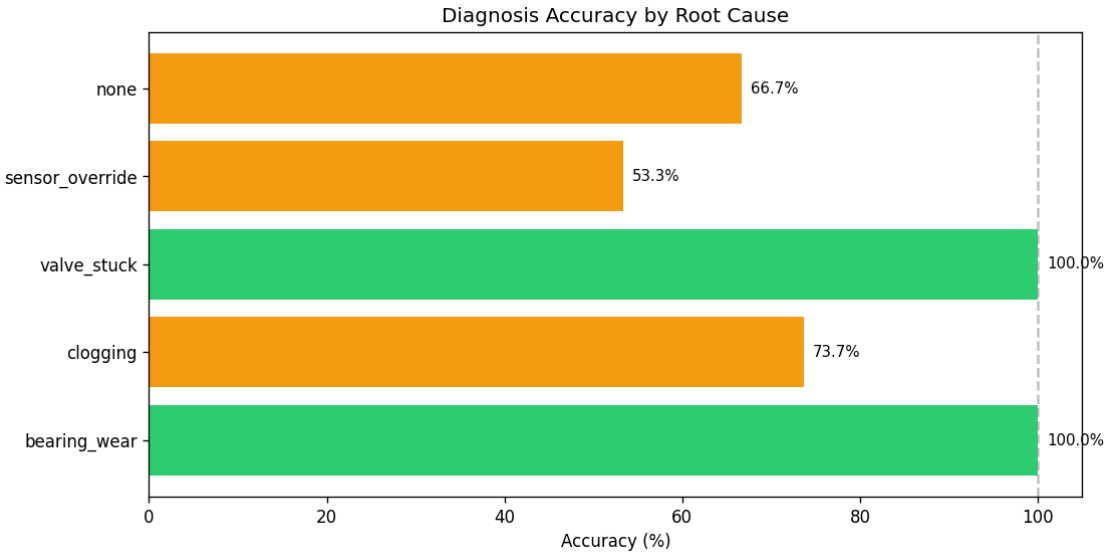

Figure 5.3: Diagnosis accuracy by root cause (%).

5.4 Scenario Matrix

Figure 5.5 compares detection rate and diagnosis accuracy per fault scenario. bearing_wear_eval and valve_flow_mismatch_eval achieve 100% on both. clogging_eval has 100% detection but 73.7% diagnosis accuracy (confusion with valve_stuck). sensor_drift_eval_temp_override and rpm_eval_override have 100% detection but lower diagnosis accuracy (63.3% and 43.3%). noise_burst_eval has 0% detection, so no diagnoses are produced.

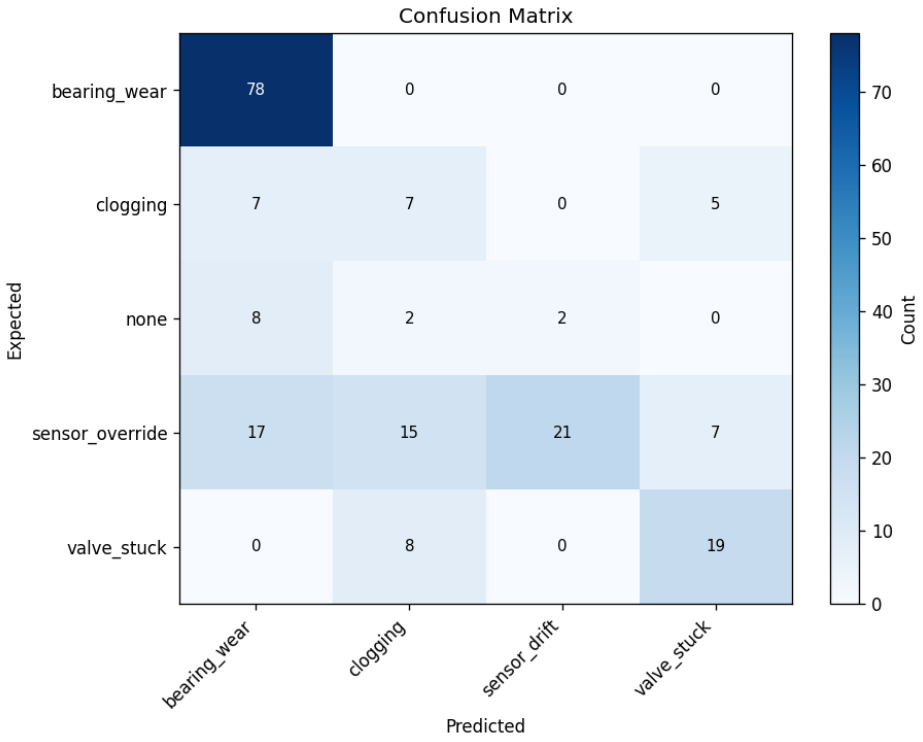

Figure 5.4: Confusion matrix: expected (rows) vs. predicted (columns).

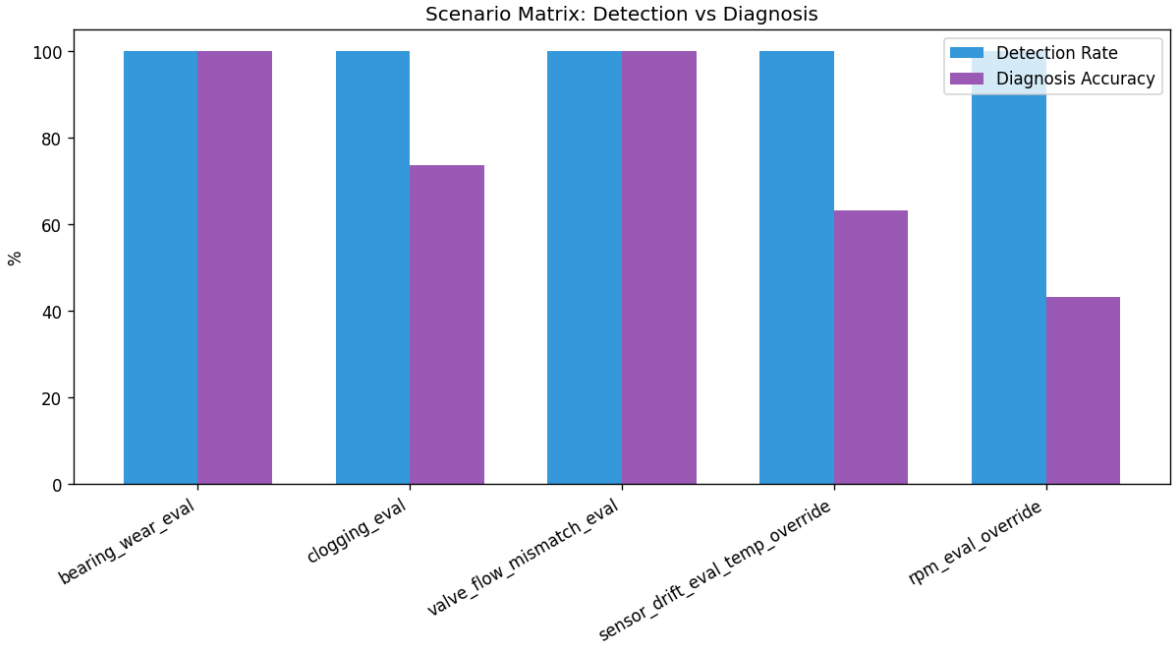

Figure 5.5: Scenario matrix: detection rate vs. diagnosis accuracy per fault scenario.

5.5 Token Usage and Latency

Correct diagnoses use fewer steps (8.2 vs. 9.3) and fewer tokens (38.5k vs. 49.8k) on average than incorrect ones (Figure 5.6). Figure 5.7 shows token usage by scenario: `bearing_wear_eval` uses the fewest (27.7k tokens); `clogging_eval`, `sensor_override` scenarios, and `healthy_base line` use more. Alert-to-diagnosis latency is low (avg 0.02 s for fault scenarios); healthy false positives have higher latency (avg 0.13 s).

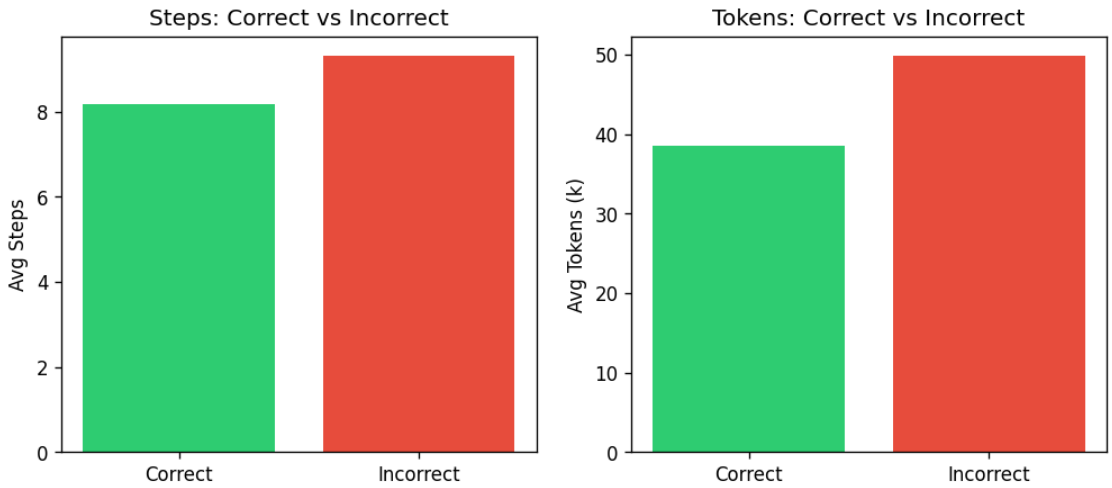

Figure 5.6: Steps and tokens: correct vs. incorrect diagnoses.

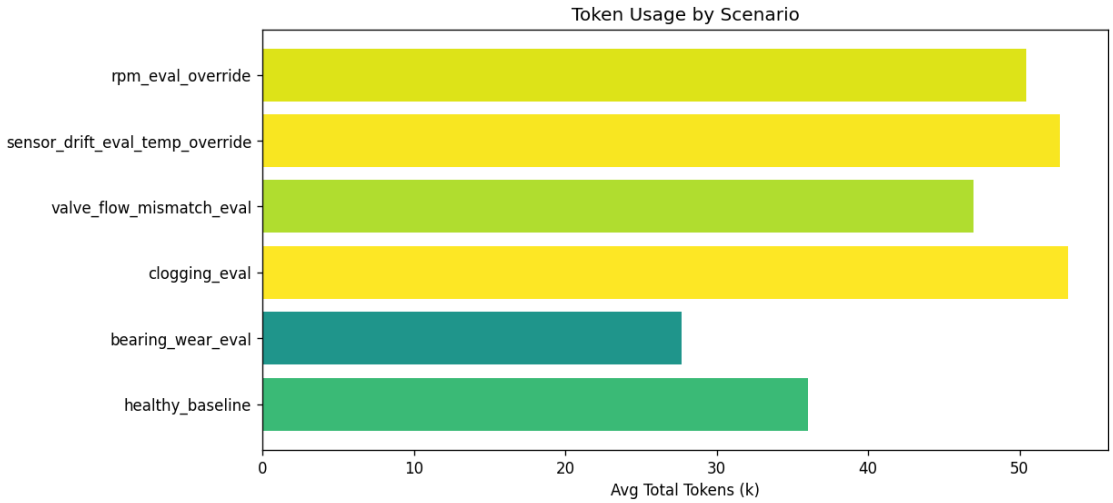

Figure 5.7: Average token usage by scenario (k tokens).

5.6 Discussion

Strengths include high accuracy for bearing wear and valve stuck (100%), interpretable evidence (ReAct reasoning and rule retrieval), and reproducible evaluation (fixed scenarios, JSONL logs, DB queries). Limitations include: (1) `noise_burst` detection failure (0%) because transient spikes do not meet duration thresholds; (2) confusion between clogging and valve_stuck (both affect flow and valve); (3) sensor_override confusion with bearing_wear and clogging (LLM struggles to distinguish sensor faults from real faults); (4) healthy false positives from `bearing_temp_c`, suggesting threshold tuning or startup filtering.

5.7 Improvement Opportunities

Alert detection: tune thresholds or reduce `min_duration_sec` for transient spikes (`noise_burst`); add startup cooldown to reduce bearing_temp_c false positives on healthy runs. Diagnosis: add discriminative rules for clogging vs. valve_stuck (e.g., slope of flow vs. valve position); improve rules for sensor_override vs. bearing_wear. RAG: batch indexing, hybrid keyword-semantic search. Future work: real sensor data, multi-asset scaling, improved noise_burst detection.

Chapter 6

Conclusion

6.1 Summary

This report presented the design, implementation, and evaluation of a multi-agent powerplant monitoring system. The architecture comprises four agents (Monitor, Diagnosis, Ticket, Review) communicating via MQTT, with separation of concerns for scalability and fault isolation. Agent A uses threshold-based and slope-based detection for real-time anomaly alerts; Agent B performs root cause diagnosis via ReAct with tool-augmented LLMs [10, 11] and rule retrieval; Agent C creates review requests; Agent D provides a human-in-the-loop interface with RAG-assisted chat [17], approve/reject workflow, and optional Salesforce Case creation. A physics-based simulator with fault injection (bearing wear, clogging, valve stuck, sensor drift, noise burst) enables reproducible evaluation.

Evaluation on 390 scenario runs showed 81.1% diagnosis accuracy with 100% for bearing wear and valve stuck, 83.3% fault scenario detection rate, and 11.9% healthy false positive rate. Correct diagnoses use fewer ReAct steps and tokens than incorrect ones; the confusion matrix reveals that sensor_override and clogging are the most challenging fault types. The system demonstrates that a rule-plus-LLM hybrid diagnosis approach with evidence traceability can achieve reliable fault diagnosis for well-defined fault types while maintaining interpretability. A key contribution of this

work is the effectiveness of the rule-plus-LLM hybrid for industrial fault diagnosis, combining interpretable rule retrieval with LLM reasoning and tool use.

6.2 Future Work

Future work includes: (1) **Vision integration**—enable VLM analysis of 3D pump renderings and dashboard screenshots for visual anomaly detection; (2) **Multi-asset scaling**—extend the pipeline to multiple pumps and assets with per-asset or shared models; (3) **Improved noise_burst detection**—tune thresholds or reduce `min_duration_sec` for transient spikes, or add a dedicated short-window detector; (4) **Real sensor data evaluation**—validate on industrial telemetry from physical pumps or test rigs; (5) **RAG enhancements**—batch indexing, hybrid keyword-semantic search, and retrieval quality tuning; (6) **Discriminative rules**—add rules to better distinguish clogging vs. valve_stuck and sensor_override vs. bearing_wear.

Bibliography

[1] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, pp. 18–23, Jan. 2015.

[2] H. Wang, W. Zhang, D. Yang, and Y. Xiang, "Deep-learning-enabled predictive maintenance in industrial internet of things: Methods, applications, and challenges," *IEEE Systems Journal*, vol. 17, no. 2, pp. 2602–2615, 2023.

[3] T. Zhu, Y. Ran, X. Zhou, and Y. Wen, "A survey on intelligent predictive maintenance (ipdm) in the era of fully connected intelligence," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 633–671, 2026.

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, pp. 1–58, July 2009.

[5] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, Jan. 2016.

[6] R. Isermann, *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Berlin, Germany: Springer, 2006.

[7] S. Yin, S. X. Ding, A. Haghani, H. Hao, and P. Zhang, "A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark tennessee eastman process," *J. Process Control*, vol. 22, pp. 1567–1581, Oct. 2012.

[8] X. Zhang, Q. Xu, and C. K. Tse, "Fault diagnosis of power electronic systems based on fault tree analysis," *IEEE Trans. Power Electron.*, vol. 24, pp. 99–112, Jan. 2009.

[9] J. Chen and R. J. Patton, *Robust Model-Based Fault Diagnosis for Dynamic Systems*. Boston, MA, USA: Kluwer, 1999.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[12] S. Yao, J. Yu, D. Zhao, I. Shafran, K. R. Narasimhan, and Y. Cao, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[13] M. Wooldridge, *An Introduction to MultiAgent Systems*. Chichester, U.K.: Wiley, 2nd ed., 2009.

[14] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Hoboken, NJ, USA: Pearson, 4th ed., 2020.

[15] OASIS, "MQTT version 3.1.1." OASIS Standard, Oct. 2014.

[16] F. Buccafurri, V. De Angelis, and S. Lazzaro, "Mqtt-i: Achieving end-to-end data flow integrity in mqtt," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4717–4734, 2024.

[17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[18] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, J. W. B. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, and C. Jones, "Improving language models by retrieving from trillions of tokens," in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 2206–2240, 2022.

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, pp. 3982–3992, 2019.

[20] A. Garcia, "sqlite-vec: Vector similarity search for SQLite," 2024.

[21] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 12888–12900, 2022.

[22] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[23] D. Zhang, Y. Yu, J. Dong, C. Li, D. Su, C. Chu, and D. Yu, "MM-LLMs: Recent advances in multimodal large language models," in *Proc. Assoc. Comput. Linguist. (ACL) Findings*, pp. 5506–5527, 2024.

[24] D. Hardt, "The OAuth 2.0 authorization framework." RFC 6749, IETF, Oct. 2012.

[25] Salesforce, "Salesforce REST API developer guide," 2024.

[26] S. Ramírez, "Fastapi: Modern, fast web framework for building APIs," 2024.

[27] LangChain, "Langchain documentation," 2024.

[28] LangChain, "Langgraph: Build resilient agents with graphs," 2024.

[29] N. Reimers and I. Gurevych, "sentence-transformers: Multilingual sentence embeddings," 2024.

[30] N. Reeh, G. Manthei, and P. J. Klar, "Physical modelling of the set of performance curves for radial centrifugal pumps to determine the flow rate," *Applied System Innovation*, vol. 6, no. 6, p. 111, 2023.

[31] F. M. White, *Fluid Mechanics*. New York, NY, USA: McGraw-Hill, 8th ed., 2016.

[32] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Chichester, U.K.: Wiley, 2011.

Appendix A

Database Schema

The schema is created by `scripts/init_db.py`. Indexes on `(asset_id,ts)` and `(ts,asset_id)` support time-range queries. The `vec_memory` virtual table is created when sqlite-vec is installed (Section 4.6). Tables are listed below with column descriptions.

telemetry

Source: Simulator. Sensor readings per timestamp.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Timestamp (ISO)
plant_id	TEXT	Plant identifier
asset_id	TEXT	Asset identifier (e.g. pump01)
pressure_bar	REAL	Pressure (bar)
flow_m3h	REAL	Flow (m^3/h)
temp_c	REAL	Temperature (°C)
bearing_temp_c	REAL	Bearing temperature (°C)
vibration_rms	REAL	Vibration RMS (mm/s)
rpm	REAL	Rotational speed
motor_current_a	REAL	Motor current (A)
valve_open_pct	REAL	Valve opening (%)
fault	TEXT	Injected fault type (default: none)
severity	REAL	Fault severity
created_at	TEXT	Row creation time

alerts

Source: Agent A. Anomaly alerts from threshold/slope detection.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Alert timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
severity	TEXT	warning, critical
signal	TEXT	Signal that triggered (e.g. bearing_temp_c)
score	REAL	Severity score
method	TEXT	threshold, slope, valve_flow_mismatch
evidence	TEXT	JSON: value, threshold, duration
created_at	TEXT	Row creation time

diagnosis

Source: Agent B. Root cause diagnosis from ReAct agent.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Diagnosis timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
alert_id	INTEGER	FK to alerts (optional)
root_cause	TEXT	bearing_wear, clogging, valve_stuck, sensor_drift, unknown
confidence	REAL	0–1
impact	TEXT	low, medium, high
recommended_actions	TEXT	JSON array
evidence	TEXT	JSON array of rule references
recursion_limit, actual_steps	INTEGER	ReAct steps (eval)
total_tokens, prompt_tokens, completion_tokens	INTEGER	Token counts (eval)
created_at	TEXT	Row creation time

vision_images

Source: Simulator. Paths to 3D renderings or screenshots.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
image_path	TEXT	File path to image
created_at	TEXT	Row creation time

vision_analysis

Source: Agent D (VLM). VLM analysis results for images.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
image_path	TEXT	Path to analyzed image
description	TEXT	VLM description
anomalies_detected	TEXT	Detected anomalies
confidence	REAL	VLM confidence
created_at	TEXT	Row creation time

tickets

Source: Agent D. External tickets (Salesforce Case, GitHub Issue) created on approve.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Creation timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
ticket_id	TEXT UNIQUE	External system ID (e.g. Case Id)
title	TEXT	Ticket title
body	TEXT	Ticket description
status	TEXT	open, closed (default: open)
diagnosis_id	INTEGER	FK to diagnosis
url	TEXT	Link to external record (e.g. Lightning URL)
created_at	TEXT	Row creation time

review_requests

Source: Agent C. Queue of diagnoses awaiting human review.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
diagnosis_id	INTEGER	FK to diagnosis
plant_id, asset_id	TEXT	Plant and asset identifiers
ts	TEXT	Timestamp
status	TEXT	pending, approved, rejected (default: pending)
created_at	TEXT	Row creation time
resolved_at	TEXT	When status was updated

chat_sessions

Source: Agent D. Chat conversation sessions.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
created_at, updated_at	TEXT	Session timestamps
preview	TEXT	Preview of last message

chat_messages

Source: Agent D. Messages within a chat session.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
session_id	INTEGER	FK to chat_sessions
role	TEXT	user, assistant
content	TEXT	Message text
tool_calls	TEXT	JSON of tool calls (assistant)
created_at	TEXT	Row creation time

chat_steps

Source: Agent D. ReAct steps (thought, tool_call, tool_result) per message.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
message_id	INTEGER	FK to chat_messages
step_type	TEXT	thought, tool_call, tool_result
step_order	INTEGER	Order within message
tool_name	TEXT	Tool name (if tool_call)
tool_args	TEXT	JSON args (if tool_call)
content	TEXT	Step content
raw_result	TEXT	Tool output (if tool_result)
created_at	TEXT	Row creation time

feedback

Source: Agent D. Human review decisions (approve/reject) with notes.

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
ts	TEXT	Timestamp
plant_id, asset_id	TEXT	Plant and asset identifiers
ticket_id	TEXT	Associated ticket ID
review_decision	TEXT	approved, rejected, edited
final_root_cause	TEXT	Reviewer-confirmed root cause
notes	TEXT	Reviewer notes
created_at	TEXT	Row creation time

vec_memory (virtual table)

Source: sqlite-vec extension. RAG vector store (384-dim embeddings).

Column	Type	Description
embedding	float[384]	Dense embedding (all-MiniLM-L6-v2)
metadata	TEXT	JSON: doc_type, doc_id, asset_id, text preview

Appendix B

Scenario JSON Example

Scenario definitions are stored as JSON files in `simulator-service/scenarios/`. The Simulator loads a scenario by name and executes the simulation loop with the specified faults, initial conditions, and setpoints. Listing B.1 shows `bearing_wear_eval.json`: a fault scenario that injects bearing wear from the start with `rate_per_sec`=0.35. Listing B.2 shows `healthy_baseline.json`: no faults, with setpoints that change valve opening at 200 s and 400 s.

Listing B.1: bearing_wear_eval.json

```
{
  "version": "1.0",
  "name": "bearing_wear_eval",
  "description": "Fast bearing wear for eval - triggers vibration/
     bearing_temp slope + threshold alerts",
  "plant_id": "plant01",
  "asset_id": "pump01",
  "seed": 12345,
  "duration_sec": 300,
  "initial_conditions": { "rpm": 2950, "valve_open_pct": 60 },
  "faults": [{ "type": "bearing_wear", "start_time_sec": 0, "params":
      { "rate_per_sec": 0.35 } }],
```

```
  "setpoints": []
}
```

Listing B.2: healthy_baseline.json

```
{
  "version": "1.0",
  "name": "healthy_baseline",
  "description": "Healthy pump operation with no faults",
  "plant_id": "plant01",
  "asset_id": "pump01",
  "seed": 12345,
  "duration_sec": 600,
  "initial_conditions": { "rpm": 2950, "valve_open_pct": 60.0 },
  "faults": [],
  "setpoints": [
    { "time_sec": 200, "valve_open_pct": 70.0 },
    { "time_sec": 400, "valve_open_pct": 50.0 }
  ]
}
```

Appendix C

Rule Document Example

Diagnosis rules are stored as Markdown files in `agent-diagnosis/rules/`. Agent B's `query_rules` tool searches these files by keyword and returns matching content. The agent uses `compute_slope` to confirm trends suggested by the rules. Listing C.1 shows `bearing_wear.md`.

Listing C.1: bearing_wear.md (agent-diagnosis/rules/)

```
# Bearing Wear


## Root Cause
bearing_wear


## Symptoms
- Elevated vibration_rms (above baseline)
- Elevated bearing_temp_c (above ambient + normal load)
- May accompany minor rpm fluctuation or motor_current_a increase


## Related Signals
vibration_rms, bearing_temp_c, rpm, motor_current_a


## Variants
```

```
- **Chronic**: vibration slope > 0 over 60s, sustained > 30s (gradual
    degradation)
- **Acute**: vibration + bearing_temp both above threshold, slope
   positive (rapid onset)

## Recommended Actions
- Check bearing lubrication level and quality
- Inspect bearing for wear, pitting, or contamination
- Schedule planned shutdown for bearing replacement if degradation
   confirmed
- Monitor vibration trend to assess progression

## Time/Trend Hints
- Use compute_slope(asset_id, "vibration_rms", 60) to check for
   gradual increase
- Use compute_slope(asset_id, "bearing_temp_c", 60) for temperature
   trend
- Chronic bearing wear: slope > 0 over 60s, sustained > 30s
- Acute: both signals above threshold with positive slope

## Impact
High - can lead to catastrophic failure if unaddressed
```

Appendix D

Project Links

The full project code and a live demo deployment are publicly available:

- **GitHub repository**: https://github.com/Haoj1/Powerplant-Multi-Agent
- **Live demo (Agent D Review Dashboard)**: https://app.powerplantagent.com/review

The GitHub repository includes installation instructions, quickstart scripts, evaluation tooling, and additional architecture documentation. The live demo exposes the Review dashboard (Review Queue, Alerts, Sensors, Chat, and Scenarios) backed by the same multi-agent system described in this report.